# libcbaia manual

**Giorgio Gonnella**

**Oct 26, 2026**

# Contents


**1 Introduction** **2**
- 1.1 Why a library . . . 2
- 1.2 Design . . . 2
- 1.3 Components . . . 3
- 1.4 What is not in the library . . . 4

**2 Building and using the library** **5**
- 2.1 Requirements . . . 5
- 2.2 Building and testing . . . 5
- 2.3 Installing the library . . . 6
- 2.4 Vendoring the library . . . 6

**3 A first program** **9**
- 3.1 The program . . . 9
- 3.2 The parts of the program . . . 11
- 3.3 Building and running . . . 12

**4 How the API works** **14**
- 4.1 Objects . . . 14
- 4.2 Ownership of the memory . . . 14
- 4.3 Errors . . . 15
- 4.4 The logger . . . 15
- 4.5 Threads . . . 15
- 4.6 Units and conventions of the values . . . 15

**5 The core component** **16**
- 5.1 Memory allocation (`alloc.h`) . . . 16
- 5.2 Logging (`logger.h`) . . . 17
- 5.3 Command line interface (`argparse-def.h`) . . . 19
- 5.4 Files and directories (`file.h`) . . . 20
- 5.5 Strings (`strutils.h`, `strbuf.h`) . . . 22
- 5.6 Numeric utilities (`numutils.h`) . . . 23
- 5.7 Progress display (`progress.h`) . . . 23
- 5.8 Other modules . . . 23

**6 Data structures** **25**

6.1 Bit operations (bitwise.h) . . . 25
6.2 Bloom filter (bloom-filter.h) . . . 25
6.3 Set of integers (set.h) . . . 25
6.4 Heap (heap.h) . . . 26
6.5 HyperLogLog (hyperloglog.h) . . . 26
6.6 Hash tables (khash) . . . 26

**7 Large files and compression** **28**
7.1 External sorting (file-sorter.h, file-merger.h) . . . 28
7.2 Parallel compression (gzip-compressor.h, bgzf-compressor.h) . . . 29
7.3 Checksums (md5.h) . . . 30
7.4 Writing HTML (html-writer.h) . . . 30
7.5 HTML reports (html-page.h) . . . 32

**8 Sequences and sequencing reads** **35**
8.1 Sequence utilities (sequtil.h) . . . 35
8.2 Reading FASTQ files (fastq-reader.h) . . . 35
8.3 Reading BAM files (bam-reader.h) . . . 36
8.4 FASTQ or BAM (reads-reader.h) . . . 36
8.5 Writing FASTQ and BAM records (fastq-writer.h, ubam-writer.h) . . . 37
8.6 The header of BAM output (sam-header.h) . . . 37
8.7 SAM tags in read titles (sam-tags.h) . . . 39
8.8 Read metadata of MinKNOW (minknow-metadata.h) . . . 39
8.9 Output files of several destinations (outfiles.h) . . . 40

**9 Barcodes, cells and library complexity** **42**
9.1 Barcode whitelists (whitelist.h) . . . 42
9.2 Edit distance of short sequences (edit-distance.h) . . . 43
9.3 Identifying the cells (cell-caller.h) . . . 44
9.4 Library complexity (complexity.h) . . . 45

**10 Recipes** **46**
10.1 A barcode demultiplexer . . . 46
10.2 Multithreaded processing . . . 51
10.3 Processing a sequencing run in real time . . . 52
10.4 A two-pass analysis of single-cell reads . . . 53

**11 Testing** **54**
11.1 Unit tests . . . 54
11.2 Program tests . . . 55
11.3 Further checks . . . 55
11.4 Fuzzing . . . 55

**12 Performance** **57**
12.1 Guidelines . . . 57
12.2 Link-time optimization . . . 57

**13 Extending the library** **58**
13.1 What belongs in the library . . . 58
13.2 Adding a module . . . 58
13.3 Moving code from a project . . . 59
13.4 Development workflow . . . 59

**14 API reference** **60**
14.1 Core (`core`) . . . . . 60
14.2 Data structures (`ds`) . . . . . 64
14.3 Large files and compression (`io`) . . . . . 65
14.4 Sequences and reads (`seq`) . . . . . 69

libcbaia is a library of C modules for **command line programs**, in particular **bioinformatics tools** working with sequencing reads: memory allocation and logging with clear error messages, a uniform command line interface, file operations, data structures, external sorting, parallel gzip and BGZF compression, FASTQ and BAM input and output, barcode correction, HTML reports and other building blocks of read processing tools.

This manual is written for **C developers** who want to use the library in their programs, or to contribute to it. The first chapters show how to build the library, how to include it in a project and how to write a first program; the chapters on the components describe each module, with examples; the last chapters describe recipes for common tasks (e.g. a demultiplexer, multithreaded processing), the tests, and how to add new modules.

# 1 Introduction

## 1.1 Why a library

Command line tools written in C, in particular in bioinformatics, need again and again the same basic functionality: allocating memory and exiting with a clear message if this fails, logging messages with different levels of verbosity, opening and reading files with descriptive error messages, parsing the command line options in a uniform way, writing unit tests, reading FASTQ and BAM files, writing compressed output in parallel, and some recurring data structures and algorithms.

Instead of copying these modules from one project to the next (and fixing the same bugs in each copy), libcbaia collects them in a single place. It is used by the C projects of the bioinformatics group (BAIA) of the Institut Pasteur du Cambodge, e.g. Lumipore, a demultiplexer of Illumina libraries sequenced on Nanopore: most of the generic code of Lumipore (FASTQ and BAM input, output files, barcode correction, cell calling, library complexity) is part of libcbaia, so that other tools can use it.

## 1.2 Design

The library follows a few principles, which make the programs using it short and robust:

**Errors exit the program, with a clear message.** Memory allocation, file operations and the parsers of the input formats do not return error codes: if something goes wrong (a file cannot be opened, the memory is exhausted, the input is invalid), the program is interrupted with a message describing the problem in terms of the user's input (file name, line, value). The caller thus does not need to check the return values, and the code stays focused on the task.

**Messages go through a logger.** The modules never write to the standard error directly: they receive a `Logger`, created by the program from the command line options (quiet, verbose, debug, log file), and use it for errors, warnings and information. The standard output is reserved for the results of the program.

**Small, independent modules.** Each module encapsulates one data structure or one task, behind a small interface: an opaque type with a constructor (`_new()`), functions operating on it and a destructor (`_delete()`). The modules have no global state, so that they can be used by several threads, each with its own objects.

**Few dependencies.** The library only needs the C standard library, POSIX and zlib (included in the repository, so that the library can be built with only a C compiler and `make`). In particular, BAM files are read and written without htslib.

**Tested.** Each module has unit tests, run under valgrind (failing on memory errors and leaks); the parsers of the input files (FASTQ, BAM, whitelists) are fuzzed.

## 1.3 Components

The modules are organized in four components (directories of `src`), shown in Fig. 1.1:

**`core`**
: the base functionality of a command line program, used by all other components: memory allocation (`alloc.h`), logging (`logger.h`), command line interface (`argparse-def.h`), files and directories (`file.h`), strings (`strutils.h`, `strbuf.h`), numeric utilities (`numutils.h`), progress display (`progress.h`), terminal colors (`colors.h`), unit tests (`test.h`);

**`ds`**
: data structures: bit operations, Bloom filter, set of integers, heap, HyperLogLog;

**`io`**
: handling of large files: sorting of files which do not fit in memory, parallel gzip and BGZF compression, MD5 checksums; self-contained HTML reports with plots;

**`seq`**
: sequences and sequencing reads: reverse complement, FASTQ and BAM reading and writing, a common reader of FASTQ and BAM files, SAM tags and MinKNOW metadata in the read titles, the header of BAM output, output files of several destinations, the edit distance of short sequences, barcode whitelists, cell calling and library complexity.

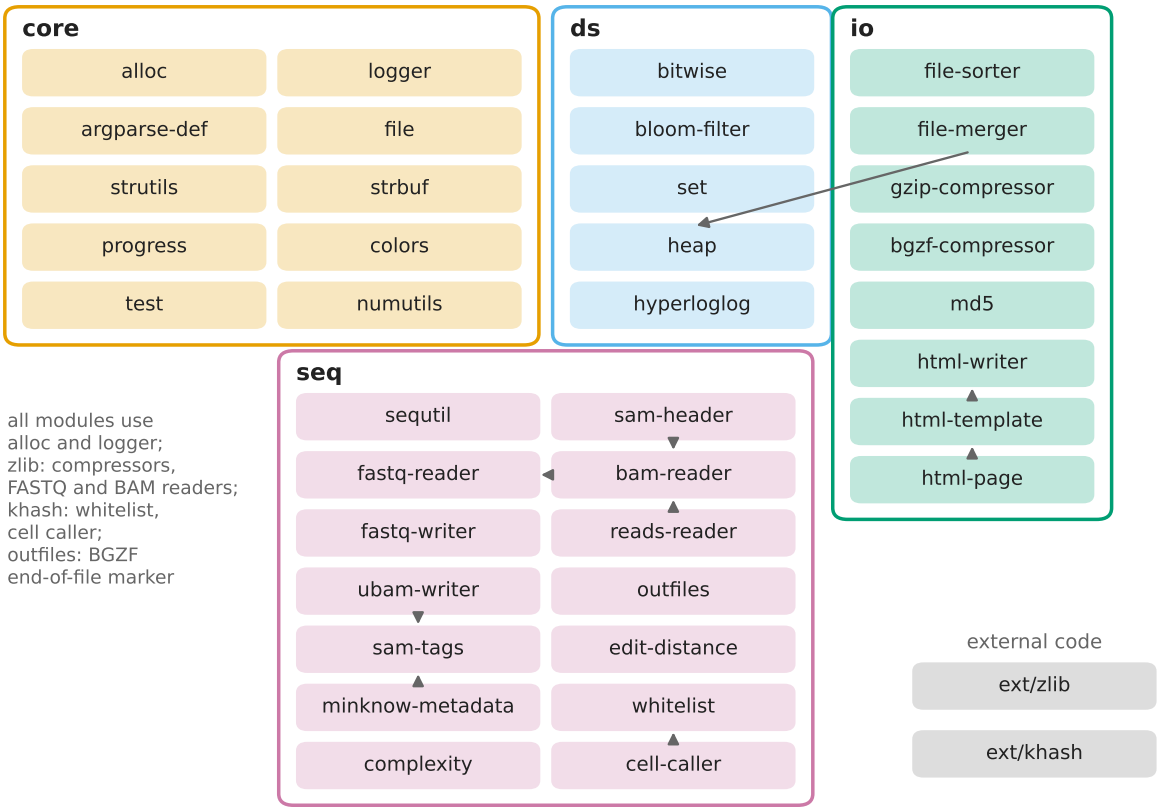



Fig. 1.1: The components and modules of libcbaia. The arrows show the main dependencies between modules (all modules use `alloc.h` and `logger.h` of `core`); external code is shown in grey.

The header names are unique over all components: the modules include each other, and are included by the programs, by their name (e.g. `#include "logger.h"`), with the directories of the components in the include path.

## 1.4 What is not in the library

The library contains generic functionality, which is useful to several programs. Code specific to one tool (e.g. the analysis of the structure of Illumina library molecules in Lumipore) stays in the tool. Code which needs further libraries (e.g. alignment with edlib, YAML parsing with libyaml) also stays in the tools which use it, so that the library keeps its minimal dependencies. *Extending the library* describes when and how to move code into the library.

# 2 Building and using the library

## 2.1 Requirements

libcbaia is written in C11 with GNU extensions (`-std=gnu11`), for POSIX systems (Linux, macOS). To build it, only a C compiler (gcc or clang) and GNU `make` are needed: zlib is included in the repository (`ext/zlib`), as is khash (`ext/khash`, the hash tables of klib, used by some modules and available to the projects). valgrind is used by the tests, if installed.

## 2.2 Building and testing

```
git clone https://github.com/baia-ipc/libcbaia
cd libcbaia
make          # lib/libcbaia.a and lib/libcbaia-zlib.a
make test     # unit tests (under valgrind) and example programs
```

The targets of the Makefile:

| | |
|---|---|
| `make` | build the static libraries `lib/libcbaia.a` and `lib/libcbaia-zlib.a` (the included zlib) |
| `make test` | build and run the unit tests (`bin/cbaia-tests`, under valgrind if available) and the example programs of this manual |
| `make fuzz` | build the fuzzing harnesses (clang; see *Testing*) |
| `make install` | install the libraries and the headers (see below) |
| `make manual` | build this manual (`manual/libcbaia-manual.pdf`; requires Sphinx and LaTeX) |
| `make clean` | remove the object and dependency files |
| `make cleanup` | also remove the libraries and the test program |

**Compilation flags.** The flags are selected with the `DEBUGFLAGS` variable; predefined sets are:

| | |
|---|---|
| DEBUGFLAGS_NDEBUG | -O3 -DNDEBUG (default) |
| DEBUGFLAGS_DEBUG | -gdwarf-3 -g3 -O0: debugging, valgrind |
| DEBUGFLAGS_PROF | -no-pie -pg and the debug flags: gprof |
| DEBUGFLAGS_SANITIZE | -fsanitize=address,undefined and the debug flags |

e.g. `make test DEBUGFLAGS='$(DEBUGFLAGS_SANITIZE)'`. When the flags change, the objects are rebuilt (the flags of the last build are saved in `obj/.flags`). Note that `-DNDEBUG` disables the `assert()` checks, which are used for programming errors only, never for checking the input.

**System zlib.** `make SYSTEM_LIBS=1` uses the zlib of the system instead of the included copy (e.g. to profit from its security updates, or when packaging the library for a distribution); the zlib header files are then required (e.g. the `zlib1g-dev` package of Debian/Ubuntu).

## 2.3 Installing the library

```
make install PREFIX=/usr/local
```

installs `libcbaia.a` and `libcbaia-zlib.a` in `$PREFIX/lib`, and the headers (with `khash.h`) in `$PREFIX/include/cbaia`. `DESTDIR` is supported for staged installations; `make uninstall` removes the installed files.

A program then includes the headers with the directory, and links the libraries:

```
#include "cbaia/logger.h"
#include "cbaia/fastq-reader.h"
```

```
cc -std=gnu11 -O2 -o prog prog.c -lcbaia -lcbaia-zlib -lm
```

(with `-lcbaia -lz -lm`, if the library was built with `SYSTEM_LIBS=1`; add `-lpthread` for multi-threaded programs). Alternatively, add `-I$PREFIX/include/cbaia` and include the headers by their name only, as in the vendored setup below.

## 2.4 Vendoring the library

The recommended way of using the library in a project is to include a copy of it in the project repository (*vendoring*), e.g. in `ext/libcbaia`: the users of the project then do not need to install the library, and each project uses a known version of it.

**Adding and updating the copy.** With `git subtree`, the copy is part of the project repository, and can be updated from the library repository:

```
# add the library to the project
git subtree add --prefix ext/libcbaia \
  https://github.com/baia-ipc/libcbaia main --squash

# update the copy to the latest version of the library
git subtree pull --prefix ext/libcbaia \
  https://github.com/baia-ipc/libcbaia main --squash
```

Changes to the library are made in the library repository (with its tests), pushed, and then pulled into the projects, not made in the copies: otherwise the copies diverge.

**The Makefile fragment.** `cbaia.mk` provides the variables and rules needed by the Makefile of the project:

```
CBAIA_DIR = ext/libcbaia
include $(CBAIA_DIR)/cbaia.mk

CFLAGS += -Wall -Wextra -std=gnu11 $(CBAIA_INCLUDES)
LDLIBS += $(CBAIA_LDLIBS) -lpthread

bin/prog: src/prog.c $(CBAIA_LIBS)
     $(CC) $(CFLAGS) $(DEBUGFLAGS) -o $@ src/prog.c $(CBAIA_LIB) $(LDLIBS)
```

The variables:

| | |
|---|---|
| `CBAIA_DIR` | the directory of the copy (set by the project) |
| `CBAIA_INCL` | the `-I` options for the headers of the components, khash and (unless `SYSTEM_LIBS=1`) zlib |
| `CBAIA_LIB` | `libcbaia.a`, to be linked |
| `CBAIA_LDLI` | the libraries to link after it: the included zlib (or `-lz`) and `-lm` |
| `CBAIA_LIBS` | all libraries built by libcbaia, as prerequisites of the programs |
| `CBAIA_EMBE` | the script `dev/embed.sh`, which writes a data file (e.g. an HTML template) as a C string literal, to be included by a source (see below) |

The library is built by its own Makefile, with the `DEBUGFLAGS` of the project, if defined, so that e.g. a debug build of the project also uses a debug build of the library. The Makefile of the library is run at each build of the project, and rebuilds the library when one of its sources, or the compilation flags, changed since the previous build (the flags are saved in `obj/.flags` of the library); otherwise, the library is left unchanged and the programs are not relinked. `make cbaia-clean` removes the build of the library.

> **Note**
>
> As the library is always built with the flags of the project, a project compiled with link-time optimization (`-flto`, as Lumipore) can inline the functions of the library, e.g. in the inner loops, also after the library was built with other flags (e.g. by `make test` in its directory): the next build of the project rebuilds it with `-flto`.

**Linking only what is used.** As `libcbaia.a` is a static library, only the modules used by a program are linked into it. In a project with several programs and modules, the script `dev/objdeps.sh` computes the object files of the *project* to link into each program, from the headers which it includes (directly or indirectly):

```
dev/objdeps.sh src obj src/prog.c   # e.g. obj/samples-info.o ...
```

so that adding a module to a project only requires creating its `.h` and `.c` files and including the header.

**Data files.** Text used by the code but better kept in its own format, e.g. an HTML template, can be embedded at compile time: `$(CBAIA_EMBED) <file>` writes the file as a C string literal (without the lines made only of a comment, `/* ... */` or `<!-- ... -->`):

```
obj/%-html.inc: src/%.html $(CBAIA_EMBED)
     $(CBAIA_EMBED) $< > $@

/* in the source (compiled with -Iobj) */
static const char *const page_tmpl =
#include "page-html.inc"
  ;
```

The library embeds its own data files (e.g. the style of the HTML pages, `src/io/html-page.css`) in the same way.

# 3 A first program

This chapter walks through a complete program using libcbaia: `readcount`, which counts the reads and bases of FASTQ or BAM files, or of the files of reads in directories (e.g. the `fastq_pass` directory of a Nanopore run). The source is in `docs/manual/examples/readcount.c` of the repository, and is built and tested by `make test`.

## 3.1 The program

```
#define VERSION "1.0"

#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "alloc.h"
#include "argparse-def.h"
#include "file.h"
#include "logger.h"
#include "reads-reader.h"

#define DEFAULT_MIN_LENGTH 0
#define BATCH_SIZE 1000

// %-PURPOSE-%
#define PURPOSE "Count the reads and bases of FASTQ or BAM files\n"\
  "\n"\
  "Usage:\n"\
  "   %s [options] <input>...\n"\
  "\n"\
  "Arguments:\n"\
  "   <input>  FASTQ (optionally gzipped) or BAM file, or a directory\n"\
  "            (its files of reads are counted)\n"\
  "\n"\
  "Options:\n"\
  "   -m, --min-length <n>  count only the reads of at least <n> bases\n"\
  "                         (default: " XSTR(DEFAULT_MIN_LENGTH) ")\n"\
```

(continues on next page)

(continued from previous page)

```
  "\n"\
  COMMON_OPTS_HELP

void parseargs(int argc, char *argv[],
    char ***inputs,
    size_t *n_inputs,
    long *min_length,
    Logger **logger)
{
  PARSE_START;
  int c = 0;
  *min_length = DEFAULT_MIN_LENGTH;
  static struct option longopts[] = {
    {"min-length", required_argument, NULL, 'm'},
    COMMON_LONGOPTS
    LONGOPTS_END
  };
  opterr = 0;
  while ((c = getopt_long(argc, argv, COMMON_OPTS_STR "m:", longopts,
                          NULL)) != -1) {
    switch (c) {
      case 'm':
        PARSE_INTEGER_OPT_MIN('m', min_length, 0);
        break;
      COMMON_OPTS_PARSE_CASES;
    }
  }
  CHECK_MIN_NARGS(1);
  *inputs = argv + optind;
  *n_inputs = (size_t)(argc - optind);
  INIT_LOGGER(*logger);
  PARSE_LOG("Inputs: %zu\n", *n_inputs);
  PARSE_LOG("Minimum read length: %ld\n", *min_length);
}

/* list_dir() filter: the files of reads */
static bool is_reads_file(const char *name, const char *path, void *data) {
  (void)data;
  return reads_reader_is_reads_filename(name) && !is_directory(path);
}

/* count the reads of a file and output a line with the counts */
static void count_file(const char *path, long min_length,
    fastq_batch_t *batch, Logger *logger) {
  reads_reader_t *reader = reads_reader_new(path, logger);
  size_t reads = 0, bases = 0, n;
  while ((n = reads_reader_read_batch(reader, batch, BATCH_SIZE)) > 0)
    for (size_t i = 0; i < n; i++) {
      size_t len = batch->records[i].seq_len;
```

(continues on next page)

(continued from previous page)

```
      if ((long)len >= min_length) {
        reads++;
        bases += len;
      }
    }
  printf("%s\t%s\t%zu\t%zu\n", path,
         reads_reader_is_bam(reader) ? "BAM" : "FASTQ", reads, bases);
  lprintf(logger, LOG_INFO, "%s: %zu reads\n", path, reads);
  reads_reader_delete(reader);
}

int main(int argc, char *argv[]) {
  char **inputs;
  size_t n_inputs;
  long min_length;
  Logger *logger;
  parseargs(argc, argv, &inputs, &n_inputs, &min_length, &logger);
  fastq_batch_t batch;
  fastq_batch_init(&batch);
  printf("file\tformat\treads\tbases\n");
  for (size_t i = 0; i < n_inputs; i++) {
    if (!is_directory(inputs[i])) {
      count_file(inputs[i], min_length, &batch, logger);
      continue;
    }
    size_t n_files;
    char **files = list_dir(inputs[i], is_reads_file, NULL, &n_files,
                            logger);
    if (n_files == 0)
      lprintf(logger, LOG_WARN, "No files of reads in %s\n", inputs[i]);
    for (size_t j = 0; j < n_files; j++) {
      count_file(files[j], min_length, &batch, logger);
      free(files[j]);
    }
    free(files);
  }
  fastq_batch_free(&batch);
  lprintf(logger, LOG_SUCCESS, "%zu inputs processed\n", n_inputs);
  delete_logger(logger);
  return EXIT_SUCCESS;
}
```

## 3.2 The parts of the program

**The help message.** `PURPOSE` is the help text, with `%s` for the name of the program. It ends with `COMMON_OPTS_HELP`, the description of the options common to all programs (verbosity, log file, help, version). The `// %-PURPOSE-%` comment marks it, so that scripts can find it (e.g. to copy the help into the documentation). `XSTR` turns the value of a macro into a string, so that the defaults in the help are always those of the code.

**Parsing the command line.** `parseargs()` follows the pattern of all programs (see *Command line interface (argparse-def.h)*):

1. `PARSE_START` declares the variables of the common options;
2. the outputs are set to their defaults;
3. the options are parsed with `getopt_long()`; the string of the short options ends with `COMMON_OPTS_STR`, the table of the long options with `COMMON_LONGOPTS` and `LONGOPTS_END`, and the `switch` with `COMMON_OPTS_PARSE_CASES`, which handle the common options; `PARSE_INTEGER_OPT_MIN` parses an integer, checking its minimum;
4. `CHECK_MIN_NARGS` checks the number of positional arguments;
5. `INIT_LOGGER` creates the logger, with the level given by the options (`-q`, `-v`, `-D`) and the log file (`-L`);
6. `PARSE_LOG` writes the values of the parameters to the log, in the verbose mode.

Errors in the options (an unknown option, a missing or invalid value, a wrong number of arguments) are reported with the usage message, and the program exits.

**Reading the files.** `reads_reader_new()` opens a FASTQ (optionally gzipped) or BAM file, recognized from its content, and `reads_reader_read_batch()` reads its records in batches (`fastq_batch_t`): the records of a batch are stored in one buffer (`batch.data`), and each record (`fastq_record_t`) gives the positions and lengths of its title, sequence and qualities in it. A batch is reused for all files, so that no memory is allocated for each record. If a file cannot be read, or is not valid, the program exits with a message giving the file and the problem.

**Directories.** `is_directory()` and `list_dir()` (`file.h`) list the files of a directory in lexicographic order, selected by a filter function, here the files of reads (`reads_reader_is_reads_filename()`: by their suffix).

**Messages.** The results go to the standard output; the messages to the logger: `lprintf(logger, LOG_INFO, ...)` is shown only in the verbose mode, `LOG_SUCCESS` and `LOG_WARN` unless `-q` is given (which leaves only the errors), `LOG_ERROR` always.

**Freeing the memory.** All objects are deleted before exiting: the programs are tested under valgrind, which reports memory leaks.

## 3.3 Building and running

The examples are built by `docs/manual/examples/Makefile`, which uses the copy of the library of the repository in the same way as a project vendoring it (see *Building and using the library*):

```
cd docs/manual/examples
make
./readcount data
```

On a terminal, the messages of the logger are colored by level (Fig. 3.1): the parameters (`-v`) in blue, the information messages in orange, the success messages in green, the errors in red. The results (the table of the counts) go to the standard output, uncolored, and can be redirected to a file without the messages.

The help, with the common options added by `COMMON_OPTS_HELP`:

```
$ ./readcount -h
Count the reads and bases of FASTQ or BAM files
```

(continues on next page)

```
$ ./readcount -v data
# 2026-10-25 02:14:43 | PARAM | Inputs: 1
# 2026-10-25 02:14:43 | PARAM | Minimum read length: 0
file    format  reads   bases
# 2026-10-25 02:14:43 | INFO | Reading data/reads.fastq
data/reads.fastq        FASTQ   8       303
# 2026-10-25 02:14:43 | INFO | data/reads.fastq: 8 reads
# 2026-10-25 02:14:43 | SUCCESS | 1 inputs processed

$ ./readcount missing.fastq
file    format  reads   bases
# 2026-10-25 02:14:43 | ERROR | Opening file 'missing.fastq' failed, No such file or directory

$ ./readcount -m abc data
# 2026-10-25 02:14:43 | ERROR | Option '-m': argument must be an integer >= 0 (abc found)
...
```

Fig. 3.1: `readcount` on a terminal: a verbose run (`-v`), a missing input file, and an invalid option value (followed by the usage, cut here). Each error interrupts the program with exit code 1.

(continued from previous page)

```
Usage:
   ./readcount [options] <input>...

Arguments:
   <input>  FASTQ (optionally gzipped) or BAM file, or a directory
            (its files of reads are counted)

Options:
   -m, --min-length <n>  count only the reads of at least <n> bases
                         (default: 0)

Verbosity options: (only one can be used)
   (default)             success, warning and error messages to stderr
   -q, --quiet           quiet: only error messages
   -v, --verbose         verbose: show additional info messages
   -D, --debug           debug: show some debug information
   -L, --logfile F       print messages to file (default: stderr)

Other options:
   -h, --help            show this help message and exit
   -V, --version         output version and exit
```

The long options (`--min-length`, `--quiet`, ...) are available because the program parses the options with `getopt_long()`, with the table of the long options ending with `COMMON_LONGOPTS` (see *Command line interface (argparse-def.h)*).

A larger example, a demultiplexer of reads by a barcode, using the barcode whitelist, the output files and the parallel compression, is described in *Recipes*.

# 4 How the API works

All modules follow the same conventions. Knowing them, the use of a module can be guessed from its header; this chapter describes them from the point of view of a program using the library. The complete coding rules (formatting, naming, commits), which also apply to the projects using the library, are in the developer guide (`docs/dev_manual.md`).

## 4.1 Objects

Most modules implement a type, whose structure is *opaque*: the header only declares it (`typedef struct whitelist_t whitelist_t;`) and the fields are private to the module. Objects are created by a constructor, used through functions taking the object as first argument, and deleted by a destructor:

```
heap_t *heap = heap_new(cmp_ints, 16);   /* constructor */
heap_push(heap, &value);                  /* operations */
int *smallest = heap_pop(heap);
heap_delete(heap);                        /* destructor */
```

The names are `<module>_new()`, `<module>_<operation>()` and `<module>_delete()`; the destructor frees everything the object owns, and accepts `NULL` (so that it can be called on objects which were not created, e.g. in cleanup code).

A few *plain data structures* are defined in the headers, as their fields are read by the callers: e.g. `fastq_batch_t` (a batch of records), `strbuf_t` (a string buffer), `hll_t` (a HyperLogLog sketch), `sam_header_inputs_t` (the read groups and programs of the inputs), `whitelist_match_t` and `complexity_t` (results). They are initialized and freed by `_init()` and `_free()` functions, or are filled by the functions that return them.

## 4.2 Ownership of the memory

The header of each function says who owns the memory of the values it returns:

- strings and arrays returned by functions such as `strcat_alloc()`, `path_join()`, `path_basename()`, `list_dir()` or `whitelist_codes()` are allocated, and must be freed by the caller (with `free()`);
- strings passed to constructors are *copied* when they are kept (e.g. the description of `progress_new()`), unless documented otherwise;

- pointers returned by accessors (e.g. `bam_reader_header()`) point into the object, and are valid as long as the object exists.

## 4.3 Errors

The library distinguishes two kinds of errors:

**Errors of the input or of the environment** (a file which cannot be opened, an invalid FASTQ record, exhausted memory, a full disk): the program is interrupted with an error message, through the logger, and the exit code `EXIT_FAILURE`. The functions therefore do not return error codes, and their results do not need to be checked: e.g. `xfopen()` always returns an open file, `fastq_reader_read_batch()` always returns valid records. The messages describe the problem in terms of the input of the user, e.g.:

```
ERROR | Opening file 'reads.fastq' failed, No such file or directory
ERROR | Whitelist barcodes.txt, line 3: barcodes of different lengths
```

The exceptions are functions whose purpose is a check, which return a boolean: e.g. `file_exists()`, `is_directory()`, `reads_reader_file_complete()`, `bam_reader_has_eof_marker()`.

**Programming errors** (a `NULL` pointer where an object is required, an index out of range): the preconditions of the exported functions are checked with `assert()`. These checks are disabled in the optimized builds (`-DNDEBUG`), so they never check the input.

As errors exit the program, the library is designed for **command line programs**; it is not suitable for long-running services or for libraries which must recover from invalid input.

## 4.4 The logger

Functions which can fail, or which report information, take a `Logger *logger` argument (usually the last one). The logger is created by the program (see *Logging (logger.h)*), from the options of the user, and passed to the modules. Any function accepts `NULL` as logger: then only the errors are output, to the standard error. This is convenient in unit tests and in small tools.

## 4.5 Threads

The modules have no global state (except constants). Different objects can thus be used by different threads at the same time; an object is not protected by locks, and is used by one thread at a time (or only read, e.g. a whitelist shared by the threads after its construction). The classes made for parallel processing are used with one object per thread (e.g. a `gzip_compressor_t` for each thread), see *Recipes*.

## 4.6 Units and conventions of the values

- Sizes are `size_t` and in bytes (e.g. in the allocation macros); counts are `size_t`; values which can be negative or need a special value (e.g. `-1` for "unknown") are `long` or `double`.
- Sequences are passed as a pointer and a length, and are not necessarily 0-terminated (e.g. a sequence in the buffer of a FASTQ batch); the functions which return 0-terminated strings say so.
- File names are often passed as a name and a suffix (`NULL` if not needed), so that the callers do not need to concatenate strings: `xfopen(basename, ".sorted", "w", "sorted output", logger)`.
- Booleans are `bool` (`stdbool.h`).

# 5 The core component

The `core` component provides the base functionality of a command line program; it is used by all other components.

## 5.1 Memory allocation (`alloc.h`)

Memory is allocated with macros which exit the program with an error message if the allocation fails, so that the result never needs to be checked. They are statements (not expressions): the pointer is the first argument.

**XMALLOC(ptr, size)**
: `ptr = malloc(size)`

**XCALLOC(ptr, nitems, itemsize)**
: `ptr = calloc(nitems, itemsize)`

**XREALLOC(ptr, size)**
: `ptr = realloc(ptr, size)`

**XFREE(ptr)**
: `free(ptr)`, then `ptr = NULL`

**XCHECKSPACE(ptrptr, capptr, req)**
: grow `*ptrptr` to `req` bytes, if its capacity `*capptr` is smaller

**XCHECKSPACEINC(ptr, capptr, req, inc)**
: grow to `req + inc` bytes, if needed

**XCHECKSPACEINCPERC(ptr, capptr, req, p)**
: grow to `req` plus `p` percent, if needed

The sizes are in bytes. `XCHECKSPACEINCPERC` is the usual way of growing a dynamic array, with amortized constant time appends:

```
int *values = NULL;
size_t n = 0, cap = 0;   /* cap: capacity in bytes */
while (...) {
  XCHECKSPACEINCPERC(values, &cap, (n + 1) * sizeof(int), 50);
  values[n++] = next_value;
```

(continues on next page)

(continued from previous page)

```
}
free(values);
```

(note that `XCHECKSPACE`, unlike the other two, takes a pointer to the buffer pointer). Compiled with `-DSHOW_TRACES`, each allocation is traced on the standard error.

## 5.2 Logging (`logger.h`)

A `Logger` writes messages to the standard error or to a log file, with a level; it shows the messages up to its own level:

| level | shown by | meaning |
|---|---|---|
| `LOG_ERROR` | always | the program is interrupted |
| `LOG_WARN` | not `-q` | a problem, but the program continues |
| `LOG_SUCCESS` | not `-q` | an operation was completed successfully |
| `LOG_PHASE` | `-v` | start or end of a phase of the program |
| `LOG_INFO` | `-v` | additional information |
| `LOG_PARAM` | `-v` | the values of the parameters |
| `LOG_MEM` | `-D` | memory usage (`showmem()`) |
| `LOG_DEBUG` | `-D` | debugging information |
| `LOG_TRACE` | (traces) | used by `TRACE()` and `DEBUG_TRACE()` |

Each message is preceded by the time and the level; at the debug levels also by the source file and line. When the messages go to a terminal, each level has its own color, so that warnings and errors stand out among the other messages; in a log file (`-L`), or when the standard error is redirected, the colors are not used.

The example program `logdemo` (`docs/manual/examples/logdemo.c`) outputs a message at each level. With the default level, only the warnings, errors and success messages are shown; `-q` shows only the errors; `-v` adds the parameters, the phases and the information messages (Fig. 5.1); `-D` adds the debugging messages, with the source file and line, and the memory usage (Fig. 5.2).

```
$ ./logdemo -p
Processing items: 100%|████████████████████| 2000000/2000000 [00:01<00:00, 1631598.68it/s]
# 2026-10-25 02:14:45 | WARNING | 3 items were skipped
# 2026-10-25 02:14:45 | SUCCESS | 2000000 items processed

$ ./logdemo -v -n 1000
# 2026-10-25 02:14:45 | PARAM | Items: 1000
# 2026-10-25 02:14:45 | PARAM | Progress display: F
# 2026-10-25 02:14:45 | PHASE | Processing the items...
# 2026-10-25 02:14:45 | INFO | Items with remainder 1 (mod 7): 143
# 2026-10-25 02:14:45 | WARNING | 3 items were skipped
# 2026-10-25 02:14:45 | SUCCESS | 1000 items processed
```

Fig. 5.1: The messages of `logdemo` with the default level (with a progress display, `-p`) and in the verbose mode (`-v`).

The code which produced these messages:

```
$ ./logdemo -D -n 1000 -e
# 2026-10-25 02:14:45 | PARAM | Items: 1000
# 2026-10-25 02:14:45 | PARAM | Progress display: F
# 2026-10-25 02:14:45 | PHASE | Processing the items...
# 2026-10-25 02:14:45 | INFO | Items with remainder 1 (mod 7): 143
# 2026-10-25 02:14:45 | DEBUG | logdemo.c:98 | Last item: 999
# 2026-10-25 02:14:45 | MEMORY | Memory usage (HWM): max=1788 kilobytes
# 2026-10-25 02:14:45 | WARNING | 3 items were skipped
# 2026-10-25 02:14:45 | ERROR | Item 500: invalid value (-e was given)
```

Fig. 5.2: All levels (`-D`): parameters (blue), phase (yellow), information (orange), debug (cyan, with the source file and line), memory usage (magenta), warning (light magenta), and the error which ends the program (red; `-e` makes the `LCHECK` of the example fail).

```
  lprintf(logger, LOG_PHASE, "Processing the items...\n");
  progress_t *progress = show_progress ?
    progress_new(stderr, "Processing items", (size_t)n_items) : NULL;
  size_t n_odd = 0;
  for (long i = 0; i < n_items; i++) {
    if (i % 7 == 1)
      n_odd++;
    if (progress != NULL && i % 1000 == 999) {
      usleep(500);   /* (the work of the example is too fast to see) */
      progress_update(progress, 1000);
    }
  }
  if (progress != NULL) {
    progress_update(progress, (size_t)(n_items % 1000));
    progress_finish(progress);
    progress_delete(progress);
  }
  lprintf(logger, LOG_INFO, "Items with remainder 1 (mod 7): %zu\n",
          n_odd);
  lprintf(logger, LOG_DEBUG, "Last item: %ld\n", n_items - 1);
  showmem(logger);
  lprintf(logger, LOG_WARN, "%zu items were skipped\n", (size_t)3);
  LCHECK(!error, logger, "Item %ld: invalid value (-e was given)\n",
         n_items / 2);
  lprintf(logger, LOG_SUCCESS, "%ld items processed\n", n_items);
```

The functions and macros:

**`Logger* new_logger(FILE *logfile, LoggingLevel level)`**
: create a logger; the programs usually create it with `INIT_LOGGER` (see below), from the options;

**`void delete_logger(Logger *logger)`**
: delete it (closing the log file, if not the standard error or output);

**`lprintf(logger, level, fmt, ...)`**
: output a message (printf-like; the format includes the final newline);

**`lnprintf(logger, level, fmt, ...)`**

continue a line started with `lprintf()`, without the prefix;

**`LCHECK(condition, logger, fmt, ...)`**
: if the condition is false, output an error message and exit with `EXIT_FAILURE` (deleting the logger, which closes the log file);

**`fatal_printf(fmt, ...)`**
: output an error and exit, where no logger is available;

**`showmem(logger)`**
: output the memory used by the program (peak resident set size), at level `LOG_MEM`;

**`TRACE(fmt, ...)`, `NTRACE(fmt, ...)`**
: temporary debugging output; `DEBUG_TRACE`/`DEBUG_NTRACE` are compiled only with `-DSHOW_TRACES`.

`LCHECK` is the usual way of checking the input and the results of system calls:

```
LCHECK(n_fields == 3, logger, "File %s, line %zu: 3 fields expected, "
       "%zu found\n", filename, lineno, n_fields);
```

All functions accept `NULL` as logger: then only errors are shown.

## 5.3 Command line interface (`argparse-def.h`)

`argparse-def.h` is a header of macros, which give all programs the same command line interface: the common options, the error messages and the help. *A first program* shows a complete example; the elements are:

**The help.** The program defines `PURPOSE` (the help text, with `%s` for the program name, ending with `COMMON_OPTS_HELP`) and, before including the header, `VERSION`. `STR(x)`/`XSTR(x)` stringify a token or the value of a macro; `BOOLSTR(b)` gives `"T"` or `"F"`.

**The common options**, handled by `COMMON_OPTS_PARSE_CASES`:

| | |
|---|---|
| `-q, --quiet` | level `LOG_ERROR`: only the error messages |
| `-v, --verbose` | level `LOG_PARAM`: information and parameters |
| `-D, --debug` | level `LOG_DEBUG` |
| `-L, --logfile` | write the messages to a file |
| `-h, --help` | output the help (`PURPOSE`) and exit |
| `-V, --version` | output the program name and `VERSION` and exit |

Only one of `-q`, `-v` and `-D` can be given; if `-L` is given more than once, the last file is used. A program which needs the letter `L` for another option defines `LOGFILE_OPT`, `LOGFILE_OPT_STR` and `LOGFILE_OPT_HELP` before including the header.

**Parsing.** In `parseargs()`:

- `PARSE_START` declares the variables of the common options;
- `getopt()` is called with `COMMON_OPTS_STR` appended to the string of the options; with `getopt_long()`, the table of the long options contains `COMMON_LONGOPTS` and ends with `LONGOPTS_END`:

```
static struct option longopts[] = {
  {"threads", required_argument, NULL, 't'},
  COMMON_LONGOPTS
  LONGOPTS_END
};
```

- the `switch` ends with `COMMON_OPTS_PARSE_CASES`; set `opterr = 0` before the loop, so that unknown options are reported by the library;
- `INIT_LOGGER(logger)` creates the logger after the parsing; `PARSE_LOG(fmt, ...)` logs the parameters (level `LOG_PARAM`).

**Values and errors.** The macros report the errors, followed by the usage, and exit:

**`PARSE_INTEGER_OPT_MIN(c, ptr, min)`**
: parse the integer value of the option `c` into `*ptr` (an integer of any type: the value is parsed as a `long`, and an error is reported if it does not fit in the type of `*ptr`), at least `min`

**`PARSE_INTEGER_OPT_MIN_MAX(c, ptr, min, max)`**
: the same, between `min` and `max`

**`PARSE_INTEGER_ARG_MIN(n, ptr, min)`**
: the same, for the `n`-th positional argument (from 0)

**`CHECK_NARGS(n)`, `CHECK_MIN_NARGS(n)`**
: check the number of positional arguments

**`OPT_ERR(c, fmt, ...)`**
: an error in the option `c`

**`ARG_ERR(n, fmt, ...)`**
: an error in a positional argument

**`PARSE_ERR(fmt, ...)`**
: another error of the command line

`argparse_long(s, &value)` parses a whole string as a decimal integer (returning `false` if it is not valid), for values which are not directly an option, e.g. the parts of an option value.

## 5.4 Files and directories (`file.h`)

The wrappers of the standard file operations have the name of the standard function with a prepended `x`; they exit the program with an error message (with the file name and the system error) if the operation fails:

| | |
|---|---|
| `xfopen(fname, sfx, mode, description, logger)` | `fopen()` |
| `xfclose(&file, logger)` | `fclose()`, sets the pointer to NULL |
| `xfread(ptr, size, nmemb, file, logger)` | `fread()`, all elements |
| `xfwrite(ptr, size, nmemb, file, logger)` | `fwrite()`, all elements |
| `xgetline(&line, &size, file, logger)` | `getline()`; false at the end of the file |
| `xgetdelim(&line, &size, delim, file, logger)` | `getdelim()` |
| `xfunlink(fname, sfx, logger)` | `unlink()` |
| `xfrename(old, sfx_old, new, sfx_new, logger)` | `rename()` |

The file name is given as a name and a suffix (`NULL` if none), which are concatenated; the description (e.g. `"input reads"`) is used in the debug messages. Further functions:

- `file_exists(fname, sfx)` and `file_size(file)`;
- `read_file(file, &buffer, &size, logger)`: read a whole file into an allocated buffer;
- `read_lines_in_buffer(file, buffer, bufsize, &lines, &linesbufsize, logger)`: read as many complete lines as fit into a buffer, and split them (used by the file sorter);
- `xfmap()`/`xfunmap()`: map a file into memory;
- `xfopen_chksig()`, `xfopen_putsig()`, `xfmap_chksig()`, `xchksig()`: binary files starting with a *signature* (a string identifying the format and its version), checked when the file is read;
- `compute_nth_filename(basename, sfx, n)`: `basename.sfx.n` (e.g. for numbered temporary files).

**Directories and paths:**

**`bool is_directory(const char *path)`**
whether the path is an existing directory;

**`void xmakedirs(const char *path, Logger *logger)`**
create a directory and its parents, if they do not exist (as `mkdir -p`);

**`char *path_join(const char *dir, const char *name)`**
the path `dir/name` (allocated), without doubling the slash if `dir` ends with one; `name` if `dir` is empty: `path_join("out/", "S1")` gives `"out/S1"`;

**`char *path_basename(const char *path)`**
the last component of a path, without trailing slashes (allocated): `"run/fastq_pass/"` gives `"fastq_pass"`;

**`char **list_dir(dir, filter, data, &n, logger)`**
the paths (`dir/name`) of the entries of a directory, in lexicographic order, without the hidden entries; `filter(name, path, data)` selects the entries (`NULL`: all). The array and the paths are allocated:

```
static bool is_subdir(const char *name, const char *path,
    void *data) {
  (void)name; (void)data;
  return is_directory(path);
}

size_t n;
char **subdirs = list_dir("fastq_pass", is_subdir, NULL, &n, logger);
for (size_t i = 0; i < n; i++) {
  process(subdirs[i]);
  free(subdirs[i]);
}
free(subdirs);
```

## 5.5 Strings (strutils.h, strbuf.h)

strutils.h:

**char *strcat_alloc(const char *s1, const char *s2)**
: the concatenation of two strings (allocated);

**char *xstrdup(const char *s)**
: a copy of a string (allocated; as `strdup()`, but exiting if the allocation fails);

**bool str_has_suffix(const char *s, const char *sfx)**
: whether `s` ends with `sfx` (and is longer than it);

**void fputs_json_string(const char *s, FILE *f)**
: write a string as a JSON string, with the quotes and the necessary escapes;

**void fputs_html_escaped(const char *s, FILE *f)**
: write a string escaping `& < > "` (e.g. user input in an HTML report);

**void format_duration(double seconds, char *buf, size_t size)**
: format a duration as `HH:MM:SS`, or `D d HH:MM:SS` from one day (rounded to the second; negative values as 0);

**bool parse_duration(const char *s, double default_unit, long *seconds)**
: parse a duration given by the user: a non-negative number, optionally followed by the unit `s`, `m` or `h` (e.g. `30s`, `1.5h`); a number without unit is in units of `default_unit` seconds (e.g. 60: minutes). The result is in seconds (rounded); `false` if the string is not valid, e.g. for an option:

```
case 'a':   /* --min-age, default unit: minutes */
  if (!parse_duration(optarg, 60, &min_age))
    OPT_ERR('a', "a duration is expected, e.g. 90s, 5m or 1.5h "
            "(%s found)", optarg);
  break;
```

**bool is_space(char c)**
: whether a character is whitespace, as for the `split()` and `strip()` of Python (ASCII: space, `\t`, `\n`, `\v`, `\f`, `\r` and the separators `\x1c`–`\x1f`); inline (it is used, e.g., by the FASTQ reader to remove the trailing whitespace of the lines).

`strbuf.h` implements a growable string buffer, `strbuf_t`, for building output text (e.g. a batch of FASTQ records) without an allocation for each piece; its memory is kept when it is cleared, so that a buffer can be reused for each batch. The content (`s`, `len`) is not 0-terminated. The functions are inline:

```
strbuf_t buf;
strbuf_init(&buf);
strbuf_append_str(&buf, "reads=");
strbuf_append_long(&buf, 42);
strbuf_append_char(&buf, '\n');
fwrite(buf.s, 1, buf.len, stdout);
strbuf_clear(&buf);           /* empty, the memory is kept */
strbuf_append(&buf, seq, len);
strbuf_free(&buf);
```

To write directly into the buffer (e.g. the output of a compressor), `strbuf_reserve(&buf, n)` returns a pointer to space for `n` characters at the end of the content, and `strbuf_commit(&buf, k)` adds the `k` characters written there to the content.

## 5.6 Numeric utilities (numutils.h)

**long floordiv(long a, long b)**
: `a / b` rounded towards minus infinity, as `//` in Python (the `/` of C rounds towards zero: `-7 / 2` is `-3`, `floordiv(-7, 2)` is `-4`); inline; `b` must not be 0. Useful for reproducing exactly the results of Python programs (see also `py_slice()` of `sequtil.h`);

**size_t hist_median(const size_t *hist, size_t n_bins, size_t bin_width, size_t total)**
: the median of the values counted in a histogram, whose bin `b` counts the values from `b * bin_width` to `(b + 1) * bin_width - 1`, with `total` values in all: the middle of the bin containing the median (the middle of the first bin if `total` is 0; the middle of the last bin if `total` exceeds the counts and the median is beyond them);

**double nice_number(double v)**
: `v` rounded up to 1, 2 or 5 times a power of 10 (at least 1), e.g. for the maximum of the axis of a plot: 130 gives 200, 4000 gives 5000.

```
size_t hist[100] = {0}, total = 0;   /* read lengths, bins of 50 bp */
for (each read) {
  size_t b = len / 50;
  hist[b < 100 ? b : 99]++;
  total++;
}
size_t median = hist_median(hist, 100, 50, total);   /* e.g. 425 */
```

## 5.7 Progress display (progress.h)

A progress display in the style of the Python `tqdm` package, on the standard error (see the first command of Fig. 5.1): a counter with the rate, or, if the total is known, a bar with the percentage and the estimated remaining time; it is refreshed at most every 0.1 seconds, overwriting the current line:

```
progress_t *p = progress_new(stderr, "Processing reads", total);
while (...)
  progress_update(p, n_new);   /* add n_new; refreshed at most every 0.1 s */
progress_finish(p);
progress_delete(p);
```

## 5.8 Other modules

- `colors.h`: ANSI colors for the terminal (`START_COLOR(stream, RED);`, `STOP_COLOR(stream);`), used by the logger; the example program `colors` writes the name of each color in that color:

```
#include <stdio.h>
#include "colors.h"

int main(void) {
  static const struct { const char *code, *name; } colors[] = {
    {BLACK, "BLACK"}, {RED, "RED"}, {GREEN, "GREEN"},
    {YELLOW, "YELLOW"}, {BLUE, "BLUE"}, {MAGENTA, "MAGENTA"},
```

(continues on next page)

(continued from previous page)

```
    {CYAN, "CYAN"}, {WHITE, "WHITE"},
    {BRIGHT_BLACK, "BRIGHT_BLACK"}, {BRIGHT_RED, "BRIGHT_RED"},
    {BRIGHT_GREEN, "BRIGHT_GREEN"}, {BRIGHT_YELLOW, "BRIGHT_YELLOW"},
    {BRIGHT_BLUE, "BRIGHT_BLUE"}, {BRIGHT_MAGENTA, "BRIGHT_MAGENTA"},
    {BRIGHT_CYAN, "BRIGHT_CYAN"}, {BRIGHT_WHITE, "BRIGHT_WHITE"},
  };
  for (size_t i = 0; i < sizeof(colors) / sizeof(colors[0]); i++) {
    START_COLOR(stdout, colors[i].code);
    printf("%s", colors[i].name);
    STOP_COLOR(stdout);
    printf("\n");
  }
  return 0;
}
```

- `test.h`: the macros of the unit tests, see *Testing*.

# 6 Data structures

The `ds` component contains data structures, which only depend on the `core` component. Hash tables are provided by khash (`khash.h`, in `ext/khash`), included with the library and available to the projects (see *Hash tables (khash)*).

## 6.1 Bit operations (`bitwise.h`)

Macros for single bits of an integer of any width: `GET_BIT(e, n)`, `SET_BIT(e, n)`, `UNSET_BIT(e, n)` (the bit number can be up to the width of the type of `e` minus 1).

## 6.2 Bloom filter (`bloom-filter.h`)

A Bloom filter of integer keys (`size_t`), of $2^k$ bytes, with two hash functions derived from the splitmix64 hash of the key. A query returns `false` if the key was certainly not inserted, `true` if it may have been:

```
bloom_filter_t *bf = bloom_filter_new(20);   /* 1 MiB */
bloom_filter_insert(bf, key);
if (bloom_filter_query(bf, other_key))
  ...   /* possibly present: check in the exact structure */
bloom_filter_delete(bf);
```

## 6.3 Set of integers (`set.h`)

A set of `size_t` values, for large numbers of elements with little memory overhead: a sorted array, queried by binary search, plus an unsorted insertion buffer, which is sorted and merged into the array when full. Optionally, the elements are divided into $2^k$ buckets (by their lowest $k$ bits), and a Bloom filter speeds up the queries of absent elements.

```
/* buffer of 4096 elements, Bloom filter of 2^16 bytes, no buckets */
Set *set = set_new(4096, 16, 0);
for (...)
  set_insert(set, value);        /* set_insert_simple(): no check of
                                    duplicates */
set_finalize_insertions(set);    /* faster queries afterwards */
if (set_query(set, x))
```

(continues on next page)

(continued from previous page)

```
  ...
set_delete(set);
```

## 6.4 Heap (heap.h)

A binary min-heap of pointers, ordered by a comparison function with the semantics of `qsort()` (called with the stored pointers): a priority queue, or the merging of sorted sequences (as in the file merger).

```
static int cmp_ints(const void *a, const void *b) {
  int x = *(const int *)a, y = *(const int *)b;
  return (x > y) - (x < y);
}

heap_t *h = heap_new(cmp_ints, 16);   /* initial capacity; it grows */
int values[] = {5, 1, 3};
for (size_t i = 0; i < 3; i++)
  heap_push(h, &values[i]);
while (!heap_is_empty(h))
  printf("%d\n", *(int *)heap_pop(h));   /* 1, 3, 5 */
heap_delete(h);                           /* not the elements */
```

## 6.5 HyperLogLog (hyperloglog.h)

The estimation of the number of distinct elements of a multiset (e.g. the distinct molecules among the reads of a sample) in a fixed amount of memory: 4 KiB ($2^{12}$ one-byte registers), with a relative standard error of about 1.6% (the estimator of Flajolet et al. 2007, with its small-range correction by linear counting; 64-bit hash values, as in Heule et al. 2013, so that no large-range correction is needed). The sketch is a plain structure, `hll_t`; sketches of parts of the data (e.g. computed by different threads) can be merged:

```
hll_t sketch;
hll_init(&sketch);
for (each read)
  hll_add(&sketch, hll_hash(key, key_len));
hll_merge(&sketch, &sketch_of_another_thread);
double distinct = hll_estimate(&sketch);
```

`hll_hash()` computes a 64-bit hash of any data; the elements are added as hash values, so that they can be of any type.

## 6.6 Hash tables (khash)

khash (from klib) implements hash maps and sets with macros, generating the code for the given key and value types. It is used, for example, by the barcode whitelist (a set of 64-bit encoded barcodes) and the cell caller (a map from barcodes to counts):

```
#include "khash.h"
```

(continues on next page)

(continued from previous page)

```
KHASH_MAP_INIT_STR(counts, size_t)   /* string -> size_t */

khash_t(counts) *h = kh_init(counts);
int absent;
khiter_t k = kh_put(counts, h, name, &absent);  /* the key is not
                                                    copied */
if (absent)
  kh_value(h, k) = 0;
kh_value(h, k)++;
k = kh_get(counts, h, "S01");
if (k != kh_end(h))
  printf("%zu\n", kh_value(h, k));
kh_destroy(counts, h);
```

Other initializers: `KHASH_MAP_INIT_INT64`, `KHASH_SET_INIT_INT64`, `KHASH_SET_INIT_STR`, `KHASH_MAP_INIT_INT` (see the header of `khash.h`). With string keys, the strings are not copied: they must live as long as the table (or be copied, and freed before `kh_destroy()`).

# 7 Large files and compression

The `io` component contains modules for handling large files: sorting files which do not fit in memory, compressing output data in parallel, and computing checksums; and the writing of the results of a program as an HTML report (with an HTML element writer).

## 7.1 External sorting (`file-sorter.h`, `file-merger.h`)

A file larger than the available memory is sorted in two steps:

1. `sort_file_parts()` reads the file in parts which fit in a buffer of a given size, splits each part into elements, sorts them, and writes them to the numbered temporary files `<out><sfx>.0`, `<out><sfx>.1`, …; it returns the number of parts;
2. `merge_n_files()` merges the sorted parts, using a heap, into the output file `<out><sfx>` (or the standard output), deleting the parts.

The elements and their order are defined by functions passed by the caller, so that any kind of record can be sorted. For text lines, the splitting function is `read_lines_in_buffer()` of `file.h`, which returns pointers to the lines in the buffer (each line ends with `\n`; the lines are not 0-terminated):

```
/* compare two lines (pointers to the line pointers) */
static int cmp_lines(const void *a, const void *b) {
  const char *x = *(char *const *)a, *y = *(char *const *)b;
  size_t lx = strcspn(x, "\n"), ly = strcspn(y, "\n");
  int c = memcmp(x, y, lx < ly ? lx : ly);
  return c != 0 ? c : (lx > ly) - (lx < ly);
}

static void sort_lines(char **lines, size_t n) {
  qsort(lines, n, sizeof(char *), cmp_lines);
}

static void output_lines(FILE *file, char **lines, size_t n,
    Logger *logger) {
  for (size_t i = 0; i < n; i++)
    xfwrite(lines[i], 1, strcspn(lines[i], "\n") + 1, file, logger);
}
```

(continues on next page)

(continued from previous page)

```
/* merging: the elements are read with xgetline() (0-terminated, with
   the newline) */
static bool get_line(char **line, size_t *size, FILE *file,
    Logger *logger) {
  return xgetline(line, size, file, logger);
}

static void write_line(FILE *file, char *line, Logger *logger) {
  (void)logger;
  fputs(line, file);
}

static int cmp_line_ptrs(const void *a, const void *b) {
  return strcmp(*(char *const *)a, *(char *const *)b);
}

/* sort big.txt into big.txt.sorted, using a buffer of 1 GiB */
size_t n_parts = sort_file_parts("big.txt", NULL, "big.txt", ".sorted",
    (size_t)1 << 30, 1024, read_lines_in_buffer, sort_lines,
    output_lines, logger);
merge_n_files(n_parts, "big.txt", ".sorted", false, get_line,
    write_line, cmp_line_ptrs, logger);
```

The buffer must be larger than the longest line (otherwise the program exits with a message suggesting a larger buffer).

## 7.2 Parallel compression (`gzip-compressor.h`, `bgzf-compressor.h`)

Compressing the output is often the most expensive part of a read processing program. The two compressors make it parallel, using a property of the gzip format: **the concatenation of gzip members is a valid gzip file**. Each block of output (e.g. the records of a batch) is compressed independently, as a gzip member, by the thread which produced it; the compressed blocks are then written to the file in the right order (see *Multithreaded processing*).

```
gzip_compressor_t *gz = gzip_compressor_new(6);   /* level 1-9 */
strbuf_t out;
strbuf_init(&out);
gzip_compressor_run(gz, text.s, text.len, &out);  /* appended to out */
fwrite(out.s, 1, out.len, file);                  /* a gzip member */
strbuf_free(&out);
gzip_compressor_delete(gz);
```

Each thread uses its own compressor (the zlib stream state is reused for all its blocks).

`bgzf-compressor.h` does the same in the BGZF format of BAM files: blocks of at most 64 KiB, each a gzip member with an extra field giving its compressed size (so that BAM readers can seek). A BGZF file ends with an empty block, the end-of-file marker `bgzf_eof` (`BGZF_EOF_LEN` bytes), written after the last block:

```
bgzf_compressor_t *bgzf = bgzf_compressor_new(6);
bgzf_compressor_run(bgzf, bam_data.s, bam_data.len, &out);
...
fwrite(bgzf_eof, 1, BGZF_EOF_LEN, file);
```

The output files of several destinations (`outfiles.h`, *Output files of several destinations (outfiles.h)*) write such compressed blocks, and add the BGZF end-of-file marker to BAM files.

## 7.3 Checksums (`md5.h`)

The MD5 digest (RFC 1321) of data or of a file, as 32 lowercase hexadecimal digits, e.g. for the checksums of files in metadata (such as the `md5` of the onlists of seqspec files) or for checking downloads; not for security purposes:

```
char hex[MD5_HEX_LEN + 1];
md5_file("barcodes.txt.gz", hex, logger);   /* as md5sum */

md5_t md5;                                  /* data in parts */
md5_init(&md5);
md5_update(&md5, data, len);
md5_update(&md5, more, more_len);
md5_final(&md5, hex);
```

## 7.4 Writing HTML (`html-writer.h`)

The markup of the HTML reports (*HTML reports (html-page.h)*), including their inline SVG plots, is written by an element writer, `html_writer_t`, which a program can also use for its own content. It escapes the text and the attribute values in one place, and indents the output, one element per line, so that the nesting is visible in the code as in the page:

```
html_writer_t w;
html_writer_init(&w, f);
html_open(&w, "table");
html_open(&w, "tr");
html_inline(&w);                          /* a row on one line */
html_tag_text(&w, "td", name);            /* escaped */
html_open(&w, "td");
html_attr(&w, "class", "%s", "seq");
html_textf(&w, "%zu", count);
html_close(&w);
html_close(&w);
html_close(&w);
html_writer_end(&w);
```

writes:

```
<table>
  <tr><td>S1 &amp; S2</td><td class="seq">42</td></tr>
</table>
```

**void html_open(html_writer_t *w, const char *tag)**
starts an element; `void html_close(html_writer_t *w)` ends the innermost open element;

**void html_elem(html_writer_t *w, const char *tag)**
an element without content and end tag: an HTML void element (e.g. `img`, `meta`) or, inside `svg`, any element (written as `<rect .../>`);

**void html_attr(html_writer_t *w, const char *name, const char *fmt, ...)**
an attribute, with the value formatted as by `printf()`, then escaped; the attributes follow `html_open()` or `html_elem()` directly: the start tag is completed by the next call of any other function. Each attribute is a separate call, so that the compiler checks the format and arguments of each value (it could not check name and value pairs passed as variable arguments);

**void html_text(html_writer_t *w, const char *text), void html_textf(html_writer_t *w, const char *fmt, ...)**
text, escaped; `void html_raw(html_writer_t *w, const char *markup)` writes trusted markup in the same way, without escaping;

**void html_tag_text(html_writer_t *w, const char *tag, const char *text), html_tag_textf()**
an element without attributes containing only text, e.g. a heading;

**void html_inline(html_writer_t *w)**
writes the content of the element just started on the same line as its tags.

An element containing text is always written on one line, with all its content (e.g. `<p>see <code>out.tsv</code></p>`), as a line break between text and inline elements would add a space in the page. `html_writer_end()` checks that all elements were closed.

**Templates** (`html-template.h`). Static markup, e.g. the skeleton of a page, is better written as an HTML file, indented normally, than as C string literals: the file is embedded in the program at compile time (see "Data files" in *Building and using the library*) and written by:

```
html_template(w, page_tmpl, "title", title, "css", css,
              "footer", footer);
```

with the values given as name and value pairs of strings. In the template, `{{name}}` is replaced by the value, escaped; `{{{name}}}` by the value as it is (trusted markup); the text between `{{#name}}` and `{{/name}}` (a section) is written only if the value is not `NULL`; a line containing only a section tag is not written:

```
<!-- (c) 2026 Name (comment lines are not embedded) -->
<title>{{title}}</title>
<style>
  {{{css}}}
</style>
{{#footer}}
<p class="note">{{footer}}</p>
{{/footer}}
```

The template is indented as the current element of the writer, and the lines of a multi-line value as the line of its placeholder (a newline at the end of the value is dropped). A placeholder without a value, a value not used, a `NULL` value outside of its section and an invalid tag (names: lower case letters, digits and _) are programming errors: the program is aborted with a message, also when compiled with `NDEBUG`, so that any test writing the template finds them.

## 7.5 HTML reports (`html-page.h`)

A program can summarize its results in an HTML report, to be opened in a browser. The pages are **self-contained**: the style is included in the page and the plots are inline SVG (JavaScript only for the reload of a report rewritten during a run), so that a report can be copied or sent as a single file, and viewed offline; the style follows the light or dark theme of the browser. A report is written by an element writer (*Writing HTML (html-writer.h)*), with which the program also writes its own content:

`void html_page_start(html_writer_t *w, const char *title, long refresh)`
: the head of the page (title, style) and the start of the body: the title as `h1` heading, and the time of writing ("Updated: …"); if `refresh` > 0, the browser reloads the page (for a report rewritten during a run, see *Processing a sequencing run in real time*), at an adaptive interval: halved, down to 1 s, when the page has changed since the last reload, doubled, up to `refresh` seconds, when not (a few lines of JavaScript; without it, every `refresh` seconds); or `void html_page_start_logo(html_writer_t *w, const char *title, const html_logo_t *logo, const char *heading, long refresh)`, a head in two columns: on the left, a logo with `heading` under it (bold spaced capitals, centered; e.g. "report", under the name shown by the logo), centered over the time of writing; on the right, the content written next (e.g. a table of the parameters), up to `void html_page_head_end(html_writer_t *w)` (on narrow screens, the right column goes below the left one). The logo (`html_logo_t`) is an image URL `src`, usually a data URI (`...`) to keep the page self-contained, an optional variant for the dark theme `src_dark` (`NULL`: none), the alternative text `alt` and the `height` in pixels;

**the content**
: written by the program with the writer (headings `h2`, paragraphs, tables), and the functions below for the tiles and the plots;

`void html_page_end(html_writer_t *w, const char *footer)`
: the end of the page, with a footer note (`NULL`: none); the elements opened by the content must have been closed.

**Reports and dashboards.** A report is a vertical page, to be read, printed or archived (e.g. at the end of an analysis). A page followed on a screen during a run is better laid out as a **dashboard**, which fits the window, so that everything is visible at once:

`void html_dashboard_start(html_writer_t *w, const char *title, const html_logo_t *logo, const char *heading, long refresh)`
: the head of the page and a top bar: the logo (`NULL`: none), with `heading` under it and the time of writing, then the content written next (e.g. the state and the parameters of the run), up to `void html_dashboard_bar_end(html_writer_t *w)`; `refresh`: as above;

**the content**
: e.g. tiles (`div.tiles`) and a grid of panels (`div.dgrid`: three columns, rows of equal height filling the window);

`void html_panel_start(html_writer_t *w, const char *title, const char *cls)`
: a panel, with a title (`h3`); `cls`: `"tall"` (two rows), `"wide"` (two columns) or `NULL`; its content scrolls if needed; `void html_panel_end(html_writer_t *w)` ends it;

`html_page_end()`
: as for a report.

On narrow (or low) windows, the panels follow each other and the page scrolls. The plots (below) are scaled to the width of the panels.

The content:

**void html_tile(html_writer_t *w, const char *value, const char *label)**
a tile with a main value and its label (text); the tiles are written inside a , which shows them in a row;

**void html_hbars(html_writer_t *w, size_t n, const char *const *labels, const size_t *values, const char *const *classes, size_t total, double mean)**
a horizontal bar chart: for each of the n values, its label, a bar and the value with its percentage of total; classes[i] is the class of the bar: "s" (main color, e.g. the samples) or "p" (muted, e.g. the unassigned reads); if mean > 0, a dashed line at that value;

**void html_histogram(html_writer_t *w, const size_t *hist, size_t n_bins, size_t bin_width, const char *what, const char *unit, const char *empty)**
a histogram of counts in bins of width bin_width (> 0; bin b: the values from b * bin_width to (b + 1) * bin_width - 1, as for hist_median()), up to the bin of the 99.5th percentile; the x axis is labelled what (unit); if all counts are 0, the note empty is written instead;

**void html_knee(html_writer_t *w, const uint64_t *counts, size_t n, size_t n_selected, const char *x_label, const char *y_label, const char *selected_label)**
a rank plot ("knee plot", log-log): counts in decreasing order (e.g. the reads of the barcodes, from cell_caller_sorted_counts()), with the axes titled x_label (log scale) and y_label (log scale), and a dashed line at the rank n_selected (0: none), labelled with the number and selected_label; e.g. for the cells of a single-cell library:

```
html_knee(w, counts, n, n_cells, "barcodes, by decreasing number "
          "of reads", "reads", "cells");
```

**html_num_t html_num(size_t n)**
a number with thousands separators (1,234,567), for the text of the report, in the string html_num(n).s: the result is a small structure returned by value (not a static buffer), valid up to the end of the statement, so that any number of them can be used in one call, e.g. html_textf(w, "%s of %s", html_num(a).s, html_num(b).s);

**double html_percent(size_t a, size_t b)**
100 * a / b (0 if b is 0).

All strings given to these functions (titles, labels, values of the tiles, notes) are text, escaped by the writer. html_page_css contains the style, e.g. for a page written otherwise; besides the headings and tables (numbers right-aligned), it defines the classes p.sub (subtitle), p.note (small, muted text), p.running (a highlighted box, e.g. "run in progress") and td.seq (sequences, in monospace), and the colors as CSS variables (e.g. --primary and --secondary, the colors of the bars of the classes s and p). The style is written in src/io/html-page.css, and the static parts of the pages (the head, the heading with the logo, the end) are templates in src/io/html-page-*.html; the Makefile of the library embeds them in html-page.c.

The example program htmlreport (docs/manual/examples/htmlreport.c) writes the report of a demultiplexer (with example values, Fig. 7.1):

```
FILE *f = xfopen(outpath, NULL, "w", "HTML report", logger);
html_writer_t w;
html_writer_init(&w, f);
html_page_start(&w, "Demultiplexing report", 0);
/* the main values, as tiles */
char value[64];
html_open(&w, "div");
```

(continues on next page)

(continued from previous page)

```
html_attr(&w, "class", "tiles");
html_tile(&w, html_num(total).s, "reads");
snprintf(value, sizeof(value), "%.1f%%", html_percent(assigned, total));
html_tile(&w, value, "assigned to a sample");
snprintf(value, sizeof(value), "%zu bp",
         hist_median(hist, N_BINS, BIN_WIDTH, n_inserts));
html_tile(&w, value, "median insert length");
html_close(&w);
/* the reads of each destination: samples (class s), unassigned (p) */
const char *classes[N_SAMPLES] = {"s", "s", "s", "p"};
html_tag_text(&w, "h2", "Reads by sample");
html_hbars(&w, N_SAMPLES, names, reads, classes, total,
           (double)assigned / (N_SAMPLES - 1));
html_open(&w, "p");
html_attr(&w, "class", "note");
html_text(&w, "Dashed line: mean of the samples.");
html_close(&w);
html_tag_text(&w, "h2", "Insert length");
html_histogram(&w, hist, N_BINS, BIN_WIDTH, "insert length", "bp",
               "No inserts.");
html_page_end(&w, "Generated by htmlreport " VERSION);
xfclose(&f, logger);
```

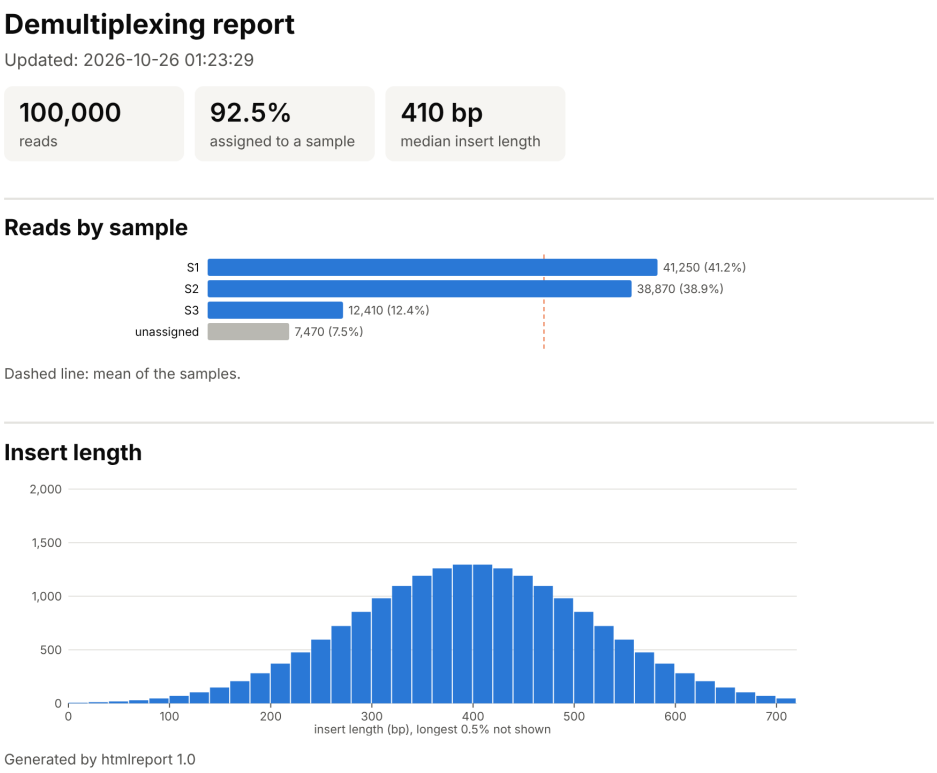



Fig. 7.1: The report written by `htmlreport`, in a browser (light theme).

# 8 Sequences and sequencing reads

The `seq` component contains the modules for sequences and for the files of sequencing reads: FASTQ (plain or gzipped) and unaligned BAM (as written by the Nanopore basecallers), for reading and writing, the SAM tags and the read metadata in the titles of the reads, the header of BAM output, and the output files of programs which distribute the reads to several destinations. The modules for barcodes (including the edit distance of short sequences), cells and library complexity are described in *Barcodes, cells and library complexity*.

## 8.1 Sequence utilities (`sequtil.h`)

**`void reverse_complement(const char *seq, size_t len, char *dest)`**
: write the reverse complement of a sequence to `dest` (at least `len + 1` bytes, 0-terminated); IUPAC codes are complemented, `U` is complemented to `A`, the case is kept and other characters are unchanged;

**`void py_slice(long len, long start, long stop, long *from, long *slice_len)`**
: the bounds of the Python slice `s[start:stop]` of a string of length `len` (negative values count from the end, values out of the range are clipped), useful for reproducing exactly the results of Python programs.

## 8.2 Reading FASTQ files (`fastq-reader.h`)

The FASTQ reader reads plain or gzipped files (recognized from the content) in **batches** of records. The parsing follows the Biopython `FastqGeneralIterator`: the title is kept as it is (except trailing whitespace), sequences and qualities can span several lines, and the title of the + line is optional, but must be equal to that of the record, if present. Invalid records interrupt the program with a message giving the file and the problem.

A batch (`fastq_batch_t`) stores all its records in one buffer (`data`); each record (`fastq_record_t`) gives the offsets and lengths of its parts in the buffer:

```
fastq_reader_t *reader = fastq_reader_new("reads.fastq.gz", logger);
fastq_batch_t batch;
fastq_batch_init(&batch);
size_t n;
while ((n = fastq_reader_read_batch(reader, &batch, 1000)) > 0) {
  for (size_t i = 0; i < n; i++) {
```

(continues on next page)

(continued from previous page)

```
    const fastq_record_t *r = &batch.records[i];
    const char *title = batch.data.s + r->title_from;  /* without @ */
    const char *seq = batch.data.s + r->seq_from;
    const char *qual = batch.data.s + r->qual_from;    /* seq_len chars */
    process(title, r->title_len, seq, qual, r->seq_len);
  }
}
fastq_batch_free(&batch);
fastq_reader_delete(reader);
```

The strings are not 0-terminated: use the lengths. A batch replaces its content at each call, and keeps its memory, so that reading a file allocates memory only for the first batches. In a multithreaded program, each thread reads its batch (under a lock) and processes it, see *Multithreaded processing*. `fastq_reader_count()` returns the number of records of a file (reading it as above, without the log message).

## 8.3 Reading BAM files (`bam-reader.h`)

The BAM reader reads BAM files (e.g. the unaligned BAM of Dorado and MinKNOW) without htslib, into the same batches of FASTQ-like records:

- the title is the read name, followed by the tags in SAM text format, separated by tabs (as in the FASTQ output of Dorado), e.g. `read1<TAB>RG:Z:run1<TAB>ch:i:42`; array tags (type `B`, e.g. the move table) and the base modification tags (`MM`, `ML`, `MN`) are not included;
- the sequence and the qualities are those of the original read (reverse complemented if the record is mapped to the minus strand); missing qualities are written as `!`;
- secondary and supplementary alignments are skipped.

`bam_reader_header()` returns the SAM header text (e.g. to copy the `@RG` lines to the output); `bam_reader_is_bam()` tells whether a file is a BAM file (by its content); `bam_reader_count()` counts the records; `bam_reader_has_eof_marker()` tells whether the file ends with the BGZF end-of-file marker, i.e. it was completely written.

## 8.4 FASTQ or BAM (`reads-reader.h`)

Most programs accept both formats. `reads_reader_t` opens either, recognized from the content, with the interface of the FASTQ reader:

```
reads_reader_t *reader = reads_reader_new(filename, logger);
while ((n = reads_reader_read_batch(reader, &batch, 1000)) > 0)
  ...
reads_reader_delete(reader);
```

Further functions:

**`bool reads_reader_is_bam(const reads_reader_t *self)`**
: whether the file read is a BAM file (e.g. to read its header with a `bam_reader_t`, or to write BAM output);

**`bool reads_reader_is_reads_filename(const char *name)`**
: whether a name is that of a file of reads: suffix `.fastq`, `.fq` (optionally `.gz`) or `.bam`; e.g. as a

filter of `list_dir()`;

**`size_t reads_reader_filename_suffix_len(const char *name)`**
the length of that suffix at the end of the name (0 if none), e.g. to remove it;

**`size_t reads_reader_count(const char *filename, Logger *logger)`**
the number of reads of a file (e.g. for the total of a progress bar); the file is read and parsed (thus also multi-line FASTQ records are counted correctly), which takes about as long as decompressing it; the program is exited if the file is not valid;

**`bool reads_reader_file_complete(const char *path)`**
whether a file written by another program is complete: a gzipped file is not truncated and ends with a newline, a FASTQ file ends with a newline, a BAM file ends with the BGZF end-of-file marker. This is used by programs which process the files of a sequencing run while they are written (e.g. the real-time mode of Lumipore), together with the age of the file:

```
struct stat st;
if (stat(path, &st) == 0 &&
    difftime(time(NULL), st.st_mtime) >= min_age &&
    reads_reader_file_complete(path))
  process(path);
```

## 8.5 Writing FASTQ and BAM records (`fastq-writer.h`, `ubam-writer.h`)

The writers append records to a `strbuf_t`, which is then compressed (with the gzip or BGZF compressor) and written to a file:

**`fastq_writer_record(&out, title, title_len, seq, qual, len)`**
a FASTQ record, in four lines (the title without `@`);

**`ubam_writer_header(&out, text)`**
the header of an unaligned BAM file, with the SAM header text (no reference sequences; see *The header of BAM output (sam-header.h)*);

**`ubam_writer_record(&out, title, title_len, seq, qual, len)`**
an unmapped BAM record (flag 4): the title contains the read name and SAM tags in text format, separated by tabs (as returned by the BAM reader); the tags are encoded in binary form (integers with the smallest type, as htslib does); fields which are not valid SAM tags are not written; names longer than `UBAM_MAX_READ_NAME` (254) are truncated.

Thus, a program can read FASTQ or BAM, and write FASTQ or BAM, keeping the tags of the input (e.g. the read group, the channel, or the tags added by the program, such as `CB` and `UB` for single-cell data).

## 8.6 The header of BAM output (`sam-header.h`)

A program which writes BAM files from BAM input (e.g. the unaligned BAM of Dorado) keeps the provenance of the reads in the SAM header text given to `ubam_writer_header()`: the read groups (`@RG`) of the inputs, so that the `RG` tags of the records still refer to a read group of the header, and the programs (`@PG`) of the inputs, followed by the `@PG` line of the program itself, linked to the last one of the inputs (`PP`), as samtools does.

The read groups and programs of the inputs are collected in a `sam_header_inputs_t` (a plain structure: `rg_ids` and `n_rg` are the IDs of the read groups, `last_pg` the ID of the last program, or `NULL`):

**void sam_header_inputs_init(sam_header_inputs_t *self), void sam_header_inputs_clear(sam_header_inputs_t *self), void sam_header_inputs_free(sam_header_inputs_t *self)**
initialize it (no read groups and programs), remove all read groups and programs, free its memory;

**void sam_header_inputs_add(sam_header_inputs_t *self, const char *text)**
add the `@RG` and `@PG` lines of a SAM header text (e.g. returned by `bam_reader_header()`), except those with an ID already added (e.g. the same read group in several files of a run);

**void sam_header_inputs_add_files(sam_header_inputs_t *self, char *const *files, size_t n_files, Logger *logger)**
the same, for the BAM files among the input files (the other files, e.g. FASTQ, are skipped).

The header text is then built with:

**void sam_header_append_read_groups(strbuf_t *text, const sam_header_inputs_t *inputs, const char *name)**
append the read groups of the inputs; with a `name` (a destination, e.g. a sample, of a program which distributes the reads), first a read group `name` (`SM:name`), then each read group `X` of the inputs renamed as `name.X`, with its fields and `SM:name`; the program then writes `RG:Z:name.X` in the records of the destination (or `RG:Z:name`, for reads without read group);

**void sam_header_append_programs(strbuf_t *text, const sam_header_inputs_t *inputs, const char *program, const char *version, const char *command_line)**
append the programs of the inputs and the `@PG` line of the program: `ID:program` (`program.1`, `.2`, ... if an input was already written by it), `PN:program`, `PP:` the last program of the inputs, `VN:version`, `CL:command_line`;

**char *sam_header_command_line(int argc, char *const argv[])**
the command line for the `CL` field: the arguments separated by spaces, with tabs and newlines replaced by spaces (allocated);

**const char *sam_header_line_id(const char *line, size_t line_len, size_t *len)**
the value of the `ID` field of a header line (not 0-terminated, its length in `*len`); `NULL` if none.

For example, a demultiplexer writing a BAM file for each sample sets the header of each destination of the output files (*Output files of several destinations (outfiles.h)*):

```
sam_header_inputs_t inputs;
sam_header_inputs_init(&inputs);
sam_header_inputs_add_files(&inputs, files, n_files, logger);
char *cl = sam_header_command_line(argc, argv);
for (size_t d = 0; d < n_destinations; d++) {
  strbuf_clear(&text);
  strbuf_append_str(&text, "@HD\tVN:1.6\tSO:unknown\n");
  sam_header_append_read_groups(&text, &inputs, names[d]);
  sam_header_append_programs(&text, &inputs, "demux", VERSION, cl);
  strbuf_append_char(&text, '\0');        /* 0-terminated */
  strbuf_clear(&bam);
  ubam_writer_header(&bam, text.s);
  strbuf_clear(&compressed);
  bgzf_compressor_run(bgzf, bam.s, bam.len, &compressed);
  outfiles_set_bam(out, d, compressed.s, compressed.len);
}
free(cl);
sam_header_inputs_free(&inputs);
```

With an input `in.bam` with the read groups `run1_A` and `run1_B` and the basecaller as last program, the header of the sample `S1` written by `demux -o out in.bam` is (fields separated by tabs):

```
@HD  VN:1.6  SO:unknown
@RG  ID:S1  SM:S1
@RG  ID:S1.run1_A  PL:ONT  SM:S1
@RG  ID:S1.run1_B  PL:ONT  SM:S1
@PG  ID:basecaller  PN:dorado  VN:0.9
@PG  ID:demux  PN:demux  PP:basecaller  VN:1.0  CL:demux -o out in.bam
```

## 8.7 SAM tags in read titles (`sam-tags.h`)

The titles of the records (FASTQ titles, the titles returned by the BAM reader, and those given to the uBAM writer) contain the read name and SAM tags in text format, separated by tabs (as in the FASTQ of Dorado, for `minimap2 -y` or `samtools import -T '*'`). Functions for building and checking them:

**`bool sam_tag_is_valid(const char *field, size_t len)`**
: whether a field is a tag in SAM text format: a two-character name (a letter and a letter or digit), a type (`A`, `i`, `f`, `Z`, `H` or `B`) and the value, separated by `:` (e.g. `RG:Z:run1`);

**`void sam_tag_append_string(strbuf_t *out, const char *tag, const char *value, size_t len)`**
: append a tab and the string tag `<tag>:Z:<value>`;

**`void sam_tag_append_int(strbuf_t *out, const char *tag, long value)`**
: append a tab and the integer tag `<tag>:i:<value>`;

**`void sam_tag_start(strbuf_t *out, const char *tag, char type)`**
: append a tab and the start of a tag (e.g. `\tQT:Z:`), for values built piece by piece (e.g. the qualities of several indices).

For example, a demultiplexer can append to the title of a read its sample barcode and the number of a molecule:

```
strbuf_append(&title, name, name_len);
sam_tag_append_string(&title, "BC", index, index_len);
sam_tag_append_int(&title, "mk", molecule);
```

## 8.8 Read metadata of MinKNOW (`minknow-metadata.h`)

The FASTQ files written by MinKNOW contain the read metadata in the title as fields `key=value`, separated by spaces (e.g. `runid=... read=12 ch=7 start_time=... flow_cell_id=... barcode=barcode01`), while Dorado writes SAM tags. `minknow_metadata_append_sam_tag(&out, field, len)` converts a field to a SAM tag (appended with a tab; returns `false`, without changing the output, for other fields):

| MinKNOW field | SAM tag | notes |
|---|---|---|
| runid | ri:Z | |
| read | rn:i | as Dorado |
| ch | ch:i | as Dorado |
| start_time | st:Z | as Dorado |
| flow_cell_id | PU:Z | standard tag (platform unit), as in the read groups of Dorado |
| protocol_group_id | pg:Z | |
| sample_id | si:Z | |
| barcode | ob:Z | MINKNOW_BARCODE_SAM_TAG (ONT barcode; Dorado writes it in BC) |
| barcode_alias | oa:Z | |
| parent_read_id | pi:Z | as Dorado |
| basecall_model_version_ | bm:Z | |

The tags which are not standard contain a lowercase letter (reserved for end users by the SAM specification) and are not used by Dorado and minimap2. The integer fields are written as Z if their value is not an integer.

## 8.9 Output files of several destinations (outfiles.h)

A demultiplexer, or any program distributing the reads to several *destinations* (e.g. the samples), writes a file for each of them. An outfiles_t manages these files:

- the destinations are numbered from 0; each has a name, used in the file name;
- for an input file, the files are <outdir>/<input basename>.<destination>.fastq.gz (the suffix of a file of reads of the input, .fastq, .fq (optionally .gz) or .bam, is removed); for a group of input files (e.g. the files of a barcode directory), <outdir>/<group>/<destination>.fastq.gz;
- a file is created when the first data are written to it, so that destinations without reads have no file;
- the data are written already compressed (gzip members, e.g. by a gzip_compressor_t), so that they can be compressed in parallel;
- with outfiles_set_append(), the data are appended to existing files (e.g. when the files of a run are processed one at a time);
- BAM output: with outfiles_set_bam(), the files are .bam instead of .fastq.gz; the data are BGZF blocks; the header (given, BGZF-compressed, for each destination) is written to each new file, and the end-of-file marker when it is closed (and removed before appending to an existing file).

```
outfiles_t *out = outfiles_new("out", "run1.fastq.gz", n_samples + 1,
                               logger);
...
/* compressed: a gzip member with the records of destination d */
outfiles_write(out, d, names[d], compressed.s, compressed.len);
...
outfiles_delete(out);   /* closes the files */
```

The output directory (and the group directory) is created if needed. *A barcode demultiplexer* shows a complete program.

# 9 Barcodes, cells and library complexity

Four modules of the `seq` component implement analyses which are common to the tools working with barcoded reads: the correction of barcodes using a whitelist, the edit distance of short sequences (e.g. for matching indices to a list), the identification of the cells of a single-cell library, and the estimate of the complexity of a library.

## 9.1 Barcode whitelists (`whitelist.h`)

A whitelist is the set of the possible barcodes of a tag: e.g. the cell barcodes of a single-cell platform (millions), the barcodes of the cells of an experiment (thousands), or the barcodes of a small multiplexing kit. The barcodes have all the same length, up to 32 bases, and are stored encoded (2 bits per base, in a `uint64_t`) in a hash set.

**Reading a whitelist.** `whitelist_new(path, logger)` reads a text file (optionally gzipped) with one barcode per line: only the first word of each line is used, without a suffix starting with `-` (such as the `-1` of the barcode files of Cell Ranger), so that e.g. a `barcodes.tsv.gz` file of Cell Ranger can be used directly. Empty lines are skipped; the barcodes must consist of the letters `ACGT`. Invalid files exit the program with a message giving the line:

```
ERROR | Whitelist cells.txt, line 3: barcodes of different lengths
```

`whitelist_new_from_codes(codes, n, length)` creates a whitelist from encoded barcodes (e.g. the cells selected by the cell caller).

**Correcting a barcode.** `whitelist_match()` searches a barcode at one end of a sequence, the *anchored* end (e.g. the end next to an adaptor): the barcode of the whitelist is aligned completely, while the sequence can be aligned partially, as the barcode is followed by other elements (e.g. a UMI). The barcode with the smallest edit distance (substitutions, insertions and deletions), up to a maximum (at most `WHITELIST_MAX_DIST`, 3), is returned if it is unique:

```
whitelist_match_t m;
/* the barcode at the start of seq (from_end false), up to 2 edits */
whitelist_match(wl, seq, len, false, 2, &m);
switch (m.status) {
  case WHITELIST_EXACT:       /* found with distance 0 */
  case WHITELIST_CORRECTED:   /* unique closest barcode, m.dist > 0 */
    whitelist_decode(wl, m.barcode, bc);   /* length + 1 bytes */
```

(continues on next page)

(continued from previous page)

```
    umi = seq + m.consumed;   /* the bases after the barcode */
    break;
  case WHITELIST_AMBIGUOUS:   /* several barcodes at the minimum
                                 distance */
  case WHITELIST_NOT_FOUND:   /* none within the maximum distance */
    break;
}
```

`m.consumed` is the length of the aligned part of the sequence: with an insertion or a deletion in the barcode, it differs from the barcode length, and the following elements start at a different position. With `from_end` true, the barcode is searched at the end of the sequence (e.g. before an adaptor).

The search enumerates, for each distance, all the sequences at that distance from the read and looks them up in the set: the time thus grows quickly with the distance, and a maximum distance of 2 is the practical limit for barcodes of 16 bases. With a large whitelist (e.g. all the barcodes of a platform), a barcode with errors is often closer to another barcode of the whitelist than to the correct one: correct the barcodes using only the barcodes of the cells (see below).

Other functions: `whitelist_encode()` and `whitelist_decode()` (between sequences and codes), `whitelist_contains()`, `whitelist_size()`, `whitelist_barcode_length()` and `whitelist_codes()` (all the codes, in an allocated array). A whitelist can be shared by several threads after its construction.

## 9.2 Edit distance of short sequences (`edit-distance.h`)

The whitelist finds the closest barcode by enumerating the sequences around the read, which is fast for a large set of barcodes and a small distance. When a sequence is instead compared with each sequence of a short list (e.g. the indices of the samples of a sequencing run, which can have different lengths), the edit distance is computed directly. `edit-distance.h` computes it for short sequences (up to `SHORT_EDIT_DISTANCE_MAX`, 64 bases), **bounded** by a maximum distance $k$: the dynamic programming is limited to the diagonal band of width $2k + 1$, and stops as soon as all the values of a row exceed $k$, so that the comparison of dissimilar sequences ends after a few rows. The functions are `static inline`: the comparison with all the sequences of a list is often in the innermost loop of a program, and is compiled in place, without a function call. For longer sequences, an alignment library (e.g. edlib) is used by the programs.

**`long short_edit_distance(const char *a, size_t alen, const char *b, size_t blen, long k)`**
: the edit distance (substitutions, insertions and deletions) of `a` and `b`, if at most `k` (`k` >= 0), otherwise `k + 1`;

**`long short_prefix_distance(const char *query, size_t qlen, const char *target, size_t tlen, long k, size_t *end)`**
: the distance of the best alignment of the whole `query` with a prefix of `target` (not empty), if at most `k`, otherwise -1; `*end` is set to the length of that prefix (the shortest among those with the best distance, as edlib in the mode `SHW`). This is the search of an element at the start of a read, followed by other elements: e.g. `short_prefix_distance("ACGT", 4, "AGTTTT", 6, 1, &end)` returns 1 (the deletion of `C`), with `end` 3.

For example, the closest index of a list, if unique:

```c
/* the index of the read matched to the indices of the samples:
   the sample with the smallest distance, up to max_dist; -1 if none
   or ambiguous */
long best = max_dist + 1, sample = -1;
for (size_t s = 0; s < n_samples; s++) {
  long d = short_edit_distance(index, index_len, samples[s].index,
                               samples[s].index_len, best);
  if (d < best) {
    best = d;
    sample = (long)s;
  } else if (d == best && d <= max_dist)
    sample = -1;   /* another sample at the same distance */
}
```

(passing the best distance found so far as the bound makes the comparisons with the following indices faster).

## 9.3 Identifying the cells (`cell-caller.h`)

In a single-cell library, most barcodes of the whitelist of the platform are not used, and the reads of the cells are mixed with background reads (e.g. of empty droplets). The cell caller identifies the cells from the counts of the barcodes found **exactly** in the reads, given the expected number of cells $N$, with the rule of Cell Ranger 2: the cells are the barcodes with a count of at least 1/10 of the 99th percentile of the counts of the $N$ most frequent barcodes (the threshold is rounded to the nearest integer, as in Cell Ranger, and is at least 1).

```c
cell_caller_t *cc = cell_caller_new();
/* first pass over the reads: count the exact barcodes */
whitelist_match(platform_wl, seq, len, false, 0, &m);
if (m.status == WHITELIST_EXACT)
  cell_caller_add(cc, m.barcode);
...
whitelist_t *cells = cell_caller_run(cc, 5000,
    whitelist_barcode_length(platform_wl), NULL,
    "out/cell_barcodes.tsv", logger);
cell_caller_delete(cc);
/* second pass: correct the barcodes using only the cells */
whitelist_match(cells, seq, len, false, 2, &m);
```

`cell_caller_run()` returns the cells as a new whitelist; if a path is given, the cells are written to it, with their counts (`barcode<TAB> count`). Cells known from another source (e.g. the cells called by Cell Ranger in an Illumina run of the same library) can be given as a whitelist (`known`): they are added to the selected cells (useful for targeted libraries, with few reads per cell). In a multithreaded program, each thread counts with its own cell caller, and the callers are merged (`cell_caller_merge()`) before the selection. `cell_caller_sorted_counts()` returns the counts in decreasing order, e.g. for plotting the "knee plot" of the barcodes.

## 9.4 Library complexity (`complexity.h`)

The complexity of a library is the number of its distinct molecules. It tells whether sequencing it more deeply is worthwhile: if most reads are duplicates, more reads will mostly be more duplicates. Without an alignment, the copies of a molecule are recognized by a *key*, e.g. the UMI and the first and last bases of the insert; the distinct keys are counted, e.g. with a HyperLogLog sketch (*Data structures*).

Sequencing errors make the copies of a molecule look distinct. With an error rate $e$ per base (e.g. measured on the alignments of known adaptor sequences), a key of length $L$ is without errors with probability $p = (1 - e)^L$; each read with an error in the key is assumed to add a distinct key. The $pN$ reads with correct keys thus contain $D = D_{obs} - (1 - p)N$ distinct molecules, and the number of molecules $M$ of the library is estimated, as in Picard `EstimateLibraryComplexity`, by solving the Lander-Waterman equation $D = M\ (1 - \exp(-*pN*/M))$.

```
complexity_t c;
complexity_estimate(n_reads, hll_estimate(&keys), mean_key_length,
                    error_rate, &c);
if (!c.estimated)
  ;   /* fewer than COMPLEXITY_MIN_READS reads with a correct key */
else if (c.saturated_unknown)
  ;   /* no duplicates: only a lower bound (c.molecules_seen) */
else
  printf("duplicates: %.1f%%, molecules: %.0f, with 2x reads: %.0f\n",
         100 * c.duplication, c.molecules, c.molecules_2x);
```

The fields of the result: `error_free` ($p$), `molecules_seen` (the distinct molecules in the reads), `duplication` (the fraction of duplicate reads), `molecules` ($M$) and `molecules_2x` (the distinct molecules expected with twice the reads). The estimate assumes molecules sampled uniformly (no amplification bias) and independent errors; with few duplicates, it is sensitive to the error of the count of the distinct keys. In the simulations of Lumipore, with 1–3% errors, the duplication was within about 0.07 of the true value, and the number of molecules within about 30% for libraries with more than 20% duplicates.

# 10 Recipes

This chapter shows how the modules work together in typical tasks of read processing programs.

## 10.1 A barcode demultiplexer

`docs/manual/examples/bcdemux.c` demultiplexes the reads of a FASTQ or BAM file by a barcode at their start: the barcode is corrected using a whitelist, removed from the read, and the read is written to the file of its barcode (or to that of the unassigned reads). In about 200 lines, the program uses the command line interface, the logger, the reads reader, the whitelist, khash, the FASTQ writer, the gzip compressor and the output files:

```
#define VERSION "1.0"

#include <stdbool.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include "alloc.h"
#include "argparse-def.h"
#include "fastq-writer.h"
#include "gzip-compressor.h"
#include "khash.h"
#include "logger.h"
#include "outfiles.h"
#include "reads-reader.h"
#include "strbuf.h"
#include "whitelist.h"

#define DEFAULT_MAX_DIST 1
#define DEFAULT_OUTDIR "."
#define BATCH_SIZE 10000

// %-PURPOSE-%
#define PURPOSE "Demultiplex reads by a barcode at their start\n"\
  "\n"\
```

(continues on next page)

(continued from previous page)

```
  "Usage:\n"\
  "   %s [options] -w <whitelist> <reads>\n"\
  "\n"\
  "Arguments:\n"\
  "   <reads>         FASTQ (optionally gzipped) or BAM file\n"\
  "\n"\
  "Options:\n"\
  "   -w, --whitelist <file>  barcodes, one per line (required)\n"\
  "   -d, --max-dist <n>      maximum edit distance of the barcodes, 0 to 3\n
↪"\
  "                           (default: " XSTR(DEFAULT_MAX_DIST) ")\n"\
  "   -o, --outdir <dir>      output directory (default: " DEFAULT_OUTDIR ")\n
↪"\
  "\n"\
  COMMON_OPTS_HELP

void parseargs(int argc, char *argv[],
    char **reads_filename,
    char **whitelist_filename,
    long *max_dist,
    char **outdir,
    Logger **logger)
{
  PARSE_START;
  int c = 0;
  *whitelist_filename = NULL;
  *max_dist = DEFAULT_MAX_DIST;
  *outdir = DEFAULT_OUTDIR;
  static struct option longopts[] = {
    {"whitelist", required_argument, NULL, 'w'},
    {"max-dist", required_argument, NULL, 'd'},
    {"outdir", required_argument, NULL, 'o'},
    COMMON_LONGOPTS
    LONGOPTS_END
  };
  opterr = 0;
  while ((c = getopt_long(argc, argv, COMMON_OPTS_STR "w:d:o:", longopts,
                          NULL)) != -1) {
    switch (c) {
      case 'w':
        *whitelist_filename = optarg;
        break;
      case 'd':
        PARSE_INTEGER_OPT_MIN_MAX('d', max_dist, 0, WHITELIST_MAX_DIST);
        break;
      case 'o':
        *outdir = optarg;
        break;
      COMMON_OPTS_PARSE_CASES;
```

(continues on next page)

(continued from previous page)

```
    }
  }
  if (*whitelist_filename == NULL)
    PARSE_ERR("the whitelist (-w) is required");
  CHECK_NARGS(1);
  *reads_filename = argv[optind];
  INIT_LOGGER(*logger);
  PARSE_LOG("Reads: %s\n", *reads_filename);
  PARSE_LOG("Whitelist: %s\n", *whitelist_filename);
  PARSE_LOG("Maximum barcode distance: %ld\n", *max_dist);
  PARSE_LOG("Output directory: %s\n", *outdir);
}

/* encoded barcode -> number of the destination */
KHASH_MAP_INIT_INT64(dest, size_t)

int main(int argc, char *argv[]) {
  char *reads_filename, *whitelist_filename, *outdir;
  long max_dist;
  Logger *logger;
  parseargs(argc, argv, &reads_filename, &whitelist_filename, &max_dist,
            &outdir, &logger);

  /* the destinations: the barcodes of the whitelist, then "unassigned" */
  whitelist_t *wl = whitelist_new(whitelist_filename, logger);
  uint64_t *codes;
  size_t n_barcodes = whitelist_codes(wl, &codes);
  size_t blen = whitelist_barcode_length(wl), unassigned = n_barcodes;
  char **names;
  XMALLOC(names, sizeof(char *) * (n_barcodes + 1));
  khash_t(dest) *dest = kh_init(dest);
  for (size_t i = 0; i < n_barcodes; i++) {
    XMALLOC(names[i], blen + 1);
    whitelist_decode(wl, codes[i], names[i]);
    int absent;
    khiter_t k = kh_put(dest, dest, codes[i], &absent);
    kh_value(dest, k) = i;
  }
  names[unassigned] = "unassigned";

  outfiles_t *outfiles = outfiles_new(outdir, reads_filename,
                                      n_barcodes + 1, logger);
  gzip_compressor_t *compressor =
    gzip_compressor_new(OUTFILES_COMPRESSLEVEL);
  strbuf_t *fastq, compressed;
  XMALLOC(fastq, sizeof(strbuf_t) * (n_barcodes + 1));
  for (size_t d = 0; d <= n_barcodes; d++)
    strbuf_init(&fastq[d]);
  strbuf_init(&compressed);
```

(continues on next page)

(continued from previous page)

```
size_t *counts;
XCALLOC(counts, n_barcodes + 1, sizeof(size_t));

reads_reader_t *reader = reads_reader_new(reads_filename, logger);
fastq_batch_t batch;
fastq_batch_init(&batch);
size_t n;
while ((n = reads_reader_read_batch(reader, &batch, BATCH_SIZE)) > 0) {
  /* the records of each destination, in FASTQ format */
  for (size_t i = 0; i < n; i++) {
    const fastq_record_t *r = &batch.records[i];
    const char *data = batch.data.s;
    whitelist_match_t m;
    whitelist_match(wl, data + r->seq_from, r->seq_len, false, max_dist,
                    &m);
    size_t d = unassigned, skip = 0;
    if (m.status == WHITELIST_EXACT || m.status == WHITELIST_CORRECTED) {
      d = kh_value(dest, kh_get(dest, dest, m.barcode));
      skip = m.consumed;
    }
    fastq_writer_record(&fastq[d], data + r->title_from, r->title_len,
                        data + r->seq_from + skip,
                        data + r->qual_from + skip, r->seq_len - skip);
    counts[d]++;
  }
  /* compressed, and written to the files of the destinations */
  for (size_t d = 0; d <= n_barcodes; d++) {
    if (fastq[d].len == 0)
      continue;
    strbuf_clear(&compressed);
    gzip_compressor_run(compressor, fastq[d].s, fastq[d].len,
                        &compressed);
    outfiles_write(outfiles, d, names[d], compressed.s, compressed.len);
    strbuf_clear(&fastq[d]);
  }
}

for (size_t d = 0; d <= n_barcodes; d++)
  if (counts[d] > 0)
    lprintf(logger, LOG_SUCCESS, "%s: %zu reads\n", names[d], counts[d]);

fastq_batch_free(&batch);
reads_reader_delete(reader);
outfiles_delete(outfiles);
gzip_compressor_delete(compressor);
for (size_t d = 0; d <= n_barcodes; d++)
  strbuf_free(&fastq[d]);
strbuf_free(&compressed);
free(fastq);
```

(continues on next page)

(continued from previous page)

```
  free(counts);
  for (size_t i = 0; i < n_barcodes; i++)
    free(names[i]);
  free(names);
  free(codes);
  kh_destroy(dest, dest);
  whitelist_delete(wl);
  delete_logger(logger);
  return EXIT_SUCCESS;
}
```

Running it on the example data (8 reads, 3 barcodes of 8 bases; Fig. 10.1): of the reads, three have a barcode with an error (a substitution, or a deletion), which is corrected; one read has no barcode of the whitelist within the maximum distance, and is written to `reads.unassigned.fastq.gz`.

```
$ ./bcdemux -v -w data/barcodes.txt -d 1 -o out data/reads.fastq
# 2026-10-25 02:14:46 | PARAM | Reads: data/reads.fastq
# 2026-10-25 02:14:46 | PARAM | Whitelist: data/barcodes.txt
# 2026-10-25 02:14:46 | PARAM | Maximum barcode distance: 1
# 2026-10-25 02:14:46 | PARAM | Output directory: out
# 2026-10-25 02:14:46 | INFO | Whitelist data/barcodes.txt: 3 barcodes of length 8
# 2026-10-25 02:14:46 | INFO | Reading data/reads.fastq
# 2026-10-25 02:14:46 | SUCCESS | TTGGCCAA: 2 reads
# 2026-10-25 02:14:46 | SUCCESS | GATTACAG: 2 reads
# 2026-10-25 02:14:46 | SUCCESS | ACGTACGT: 3 reads
# 2026-10-25 02:14:46 | SUCCESS | unassigned: 1 reads

$ ls out
reads.ACGTACGT.fastq.gz  reads.TTGGCCAA.fastq.gz
reads.GATTACAG.fastq.gz  reads.unassigned.fastq.gz
```

Fig. 10.1: `bcdemux` on the example data, in the verbose mode: the parameters, the whitelist read (`whitelist_new()` reports its size), the input file opened by the reads reader, the counts, and the output files.

The structure is that of most read processing programs:

1. the destinations are set up: here the barcodes of the whitelist plus `unassigned`; a khash map gives the destination of each encoded barcode;
2. the reads are read in batches; for each read, the destination is computed, and the output record is appended to the buffer (`strbuf_t`) of its destination;
3. after each batch, the buffer of each destination is compressed (a gzip member) and written to its file by `outfiles_write()`; the buffers are cleared and reused for the next batch.

No memory is allocated for each read: the batch, the buffers of the destinations and the compressor are reused. With BAM input, the program can also write BAM output (`outfiles_set_bam()`, `ubam-writer.h`), keeping the tags of the reads; the header of each output file is built with `sam-header.h` from the read groups and programs of the inputs (*The header of BAM output (sam-header.h)*). The same structure, with the work of step 2 and the compression of step 3 done by several threads, gives a parallel program.

## 10.2 Multithreaded processing

The modules have no global state, and the compressors produce independent gzip members: a read processing program is parallelized by giving each thread its own objects (batch, output buffers, compressor, counters) and synchronizing only the input and the output (Fig. 10.2):

- **input**: a thread takes the lock of the reader, reads the next batch and its number, and releases the lock;
- **processing**: the thread processes its batch and compresses the output of each destination, without any lock (this is where the time is spent);
- **output**: the thread waits until the batches before its own have been written (a counter of the next batch to write, and a condition variable), writes its compressed blocks, merges its counters into the totals, increments the counter and wakes the other threads.

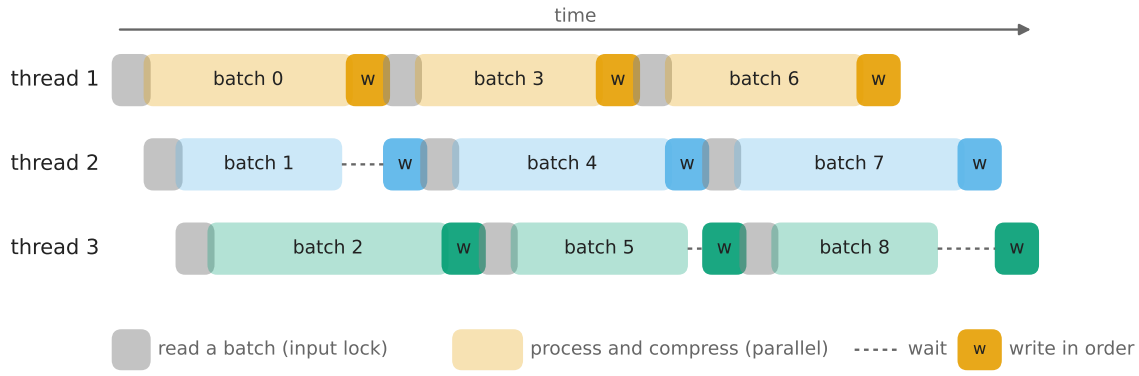



Fig. 10.2: Multithreaded processing of a file of reads: the threads read the batches in turn, process and compress them in parallel, and write them in the order of the input.

As the output is written in the order of the input, it does not depend on the number of threads: the same input gives byte-identical output files with 1 or 16 threads, which makes the program testable. The skeleton of a worker thread:

```
typedef struct {
  reads_reader_t *reader;
  pthread_mutex_t input_mutex, output_mutex;
  pthread_cond_t output_cond;
  size_t next_batch, next_to_write;
  outfiles_t *outfiles;
} job_t;

typedef struct {
  job_t *job;
  fastq_batch_t batch;
  strbuf_t *outputs, *compressed;   /* one for each destination */
  gzip_compressor_t *compressor;
  size_t n_destinations;
} worker_t;

static void *worker_run(void *data) {
```

(continues on next page)

(continued from previous page)

```
  worker_t *w = data;
  job_t *job = w->job;
  for (;;) {
    /* input */
    pthread_mutex_lock(&job->input_mutex);
    size_t n = reads_reader_read_batch(job->reader, &w->batch, 10000);
    size_t batch_num = job->next_batch++;
    pthread_mutex_unlock(&job->input_mutex);
    if (n == 0)
      break;
    /* processing and compression, in parallel */
    for (size_t i = 0; i < n; i++)
      process_record(w, &w->batch.records[i]);   /* -> w->outputs[d] */
    for (size_t d = 0; d < w->n_destinations; d++) {
      strbuf_clear(&w->compressed[d]);
      if (w->outputs[d].len > 0)
        gzip_compressor_run(w->compressor, w->outputs[d].s,
                            w->outputs[d].len, &w->compressed[d]);
      strbuf_clear(&w->outputs[d]);
    }
    /* output, in the order of the batches */
    pthread_mutex_lock(&job->output_mutex);
    while (job->next_to_write != batch_num)
      pthread_cond_wait(&job->output_cond, &job->output_mutex);
    for (size_t d = 0; d < w->n_destinations; d++)
      if (w->compressed[d].len > 0)
        outfiles_write(job->outfiles, d, name(d), w->compressed[d].s,
                       w->compressed[d].len);
    job->next_to_write++;
    pthread_cond_broadcast(&job->output_cond);
    pthread_mutex_unlock(&job->output_mutex);
  }
  return NULL;
}
```

(the batch number is taken also when the end of the file is reached, so that the numbers of the batches with data are consecutive). The worker of Lumipore (`src/worker.c` of its repository) follows this scheme, and processes about 60,000 Nanopore reads per second with 4 threads.

Statistics are accumulated by each thread in its own counters, merged under the output lock; the same holds for the sketches (`hll_merge()`) and the cell callers (`cell_caller_merge()`).

## 10.3 Processing a sequencing run in real time

A sequencer writes the files of reads while the run progresses (e.g. MinKNOW writes a file every few minutes in each `fastq_pass/barcodeNN` directory). A program processing them in real time polls the directories, and processes each new file once it is complete:

```
static bool reads_file(const char *name, const char *path, void *d) {
  (void)d;
```

(continues on next page)

(continued from previous page)

```
  return reads_reader_is_reads_filename(name) && !is_directory(path);
}

for (;;) {
  size_t n;
  char **files = list_dir(dir, reads_file, NULL, &n, logger);
  for (size_t i = 0; i < n; i++) {
    struct stat st;
    if (!already_processed(files[i]) &&
        stat(files[i], &st) == 0 &&
        difftime(time(NULL), st.st_mtime) >= MIN_AGE &&
        reads_reader_file_complete(files[i]))
      process(files[i]);   /* e.g. with outfiles_set_append() */
    free(files[i]);
  }
  free(files);
  sleep(10);
}
```

`reads_reader_file_complete()` checks the content (a complete gzip stream ending with a newline, or a BAM file with the end-of-file marker), so that a file which is still being written is not processed; the minimum age protects against files whose writing pauses. The output files are opened in append mode (`outfiles_set_append()`), so that the reads of each new file are appended to the outputs. Durations given by the user (e.g. the minimum age, or the time after which the program stops if no new files arrive) are parsed with `parse_duration()` (`strutils.h`), which accepts e.g. `90s`, `5m` or `1.5h`. A report of the results so far can be rewritten after each file, with `html_page_start()` called with a `refresh` interval, so that the page open in the browser follows the run (*HTML reports (html-page.h)*): it is reloaded often while it changes, and less often when the run is quiet.

## 10.4 A two-pass analysis of single-cell reads

For single-cell libraries, the barcodes are best corrected using the barcodes of the cells, which are not known in advance. The cell caller (*Barcodes, cells and library complexity*) is used in a first pass over the reads, counting the barcodes of the whitelist of the platform found exactly; the cells are then selected, and the reads are processed in a second pass, correcting the barcodes using the cells only. The first pass needs only the barcodes (e.g. the first bases after an adaptor), and can be made parallel in the same way as the processing, with a cell caller for each thread, merged at the end. The counts of the barcodes, in decreasing order (`cell_caller_sorted_counts()`), and the number of cells give the barcode rank plot of the report (`html_knee()`).

# 11 Testing

The library and the projects using it are tested at three levels: unit tests of the modules, tests of the programs on small input files, and further checks (sanitizers, static analysis, fuzzing).

## 11.1 Unit tests

Each module with non-trivial logic exports a `<module>_test()` function (e.g. `whitelist_test(tmpdir)`; with a directory for temporary files, if it needs files), using the macros of `test.h`:

**`test_assert(cond)`**
: exit with an error message (file, line, condition) if the condition is false

**`test_eq(found, exp, type, fmt)`**
: compare two values of a type; output `[OK]` or the differing values

**`test_eq_size`, `test_eq_int`, `test_eq_long`, `test_eq_char`**
: shortcuts of `test_eq` for `size_t`, `int`, `long`, `char`

**`test_eq_str(found, exp)`**
: compare two strings

e.g.:

```
void heap_test(void) {
  heap_t *h = heap_new(cmp_ints, 1);
  int v[] = {3, 1, 2};
  for (size_t i = 0; i < 3; i++)
    heap_push(h, &v[i]);
  test_eq_size(heap_size(h), 3);
  test_eq_int(*(int *)heap_pop(h), 1);
  heap_delete(h);
}
```

The tests of the library are run by `bin/cbaia-tests` (source `tests/cbaia-tests.c`), with one option for each group of tests, so that a group can be run alone (`bin/cbaia-tests -w`: the whitelist); without options, all tests are run. `make test` runs it under valgrind (`--error-exitcode=1 --leak-check=full --errors-for-leak-kinds=definite`): memory errors and leaks make the

tests fail. A new module adds its test function to the table of the test groups, with an option letter which is not used by the common options (`q v D L h V`) or by another group.

The projects have their own test program (e.g. `bin/lp-tests` of Lumipore), with the same structure.

## 11.2 Program tests

The programs are tested on small input files, comparing their output with the expected results. `scripts/test-cmd-lib.sh` provides the `run_cmp` bash function, for the test targets of a Makefile:

```
source scripts/test-cmd-lib.sh
run_cmp "readcount" "valgrind -q --error-exitcode=1" \
  "bin/readcount testdata/reads.fastq" 1 \
  out/readcount.tsv testdata/readcount.tsv.expected 0
```

The arguments: a label, a prefix of the command (e.g. valgrind), the command, whether its standard output is redirected to the first result file, then up to three pairs of result and expected files (the first pair with a flag: compare after sorting). The function checks the exit code and compares the files; on differences, the output is copied to `<expected>.found`, for inspection (and, if correct, to become the new expected file).

The example programs of this manual are tested in the same spirit by `make -C docs/manual/examples check` (part of `make test`).

When a program reimplements an existing tool (e.g. a prototype in Python), the expected results are produced by that tool: the C program must produce byte-identical outputs on the same inputs.

## 11.3 Further checks

Before a release, and after changes to the parsers or to the memory handling:

- **Sanitizers**: build and run the tests with `DEBUGFLAGS='$(DEBUGFLAGS_SANITIZE)'` (address and undefined behaviour sanitizers).
- **Warnings**: the code compiles without warnings with `-Wall -Wextra`, with gcc and clang (`make CPPFLAGS=-Werror`).
- **Static analysis**: e.g. `clang-tidy --checks='clang-analyzer-*,bugprone-*'`, reviewing the findings (some are false positives, e.g. in khash).
- **Fuzzing**: see below.

## 11.4 Fuzzing

The parsers of the input files are fuzzed with libFuzzer (clang, with the address and undefined behaviour sanitizers; on Debian/Ubuntu, the clang runtime package, e.g. `libclang-rt-18-dev`, is needed). The harnesses are in `tests/fuzz`:

**`fuzz-reads.c`**
: files of reads, through the reads reader (FASTQ, gzipped FASTQ or BAM): the records are read in small batches, written again as FASTQ and as unaligned BAM, and reverse complemented; the reads are counted and the completeness of the file is checked;

**`fuzz-bam.c`**
: BAM files: header and records, encoded again as unaligned BAM;

`fuzz-whitelist.c`
: barcode whitelists: barcodes searched at both ends with distances 0 to 2, and cells identified from the exact matches.

```
make fuzz                          # builds bin/fuzz/fuzz-<harness>
tests/fuzz/run.sh reads 600 4      # harness, seconds, parallel jobs
```

The corpus is kept in `obj/fuzz/corpus-<harness>` (started from the seeds in `tests/fuzz/seeds/<harness>`); crashing inputs are written to `obj/fuzz/crashes-<harness>/` and can be replayed with `bin/fuzz/fuzz-<harness> <file>`.

As the parsers exit the program on invalid input, `exit()` is wrapped (linker option `--wrap=exit`) so that it returns to the harness (`FUZZ_RUN()` of `tests/fuzz/fuzz-common.h`), which closes the files left open; leak detection is disabled, as the memory of an interrupted parse is not freed. `fuzz-common.h` is also used by the harnesses of the projects, which fuzz their own parsers (e.g. the sample sheets and library configurations of Lumipore): they add `tests/fuzz` of the library to the include path, and define `FUZZ_NAME` (used in the name of the temporary input file).

# 12 Performance

Read processing programs handle millions of records; the library is designed so that the programs using it spend their time in their own work, not in memory management, input or output.

## 12.1 Guidelines

**No allocations in the inner loops.** The batches of records, the output buffers (`strbuf_t`) and the compressors are allocated once (per thread) and reused: they keep their memory when cleared, and grow only when a larger record arrives. The memory usage does not depend on the size of the input (e.g. Lumipore uses 20–50 MB, whatever the size of the run).

**Batches.** The readers return batches of records stored in one buffer, which amortizes the cost of the input and of the locking in a multithreaded program (a batch of 1,000–10,000 records is a good size).

**Parallel compression.** Writing gzipped output is often the most expensive step: with the gzip and BGZF compressors, each thread compresses its own output (*Multithreaded processing*).

**Cache-friendly structures.** Arrays with `len` and `cap` fields are preferred to linked structures; barcodes are encoded in 64-bit integers (2 bits per base) and looked up in hash sets; the HyperLogLog sketches count distinct elements in 4 KiB.

**No logging in the inner loops.** Messages are logged per file or per phase, not per record; `TRACE` and `DEBUG_TRACE` are for temporary debugging only.

**Measure before optimizing.** Use realistic inputs, and compare the versions with the same data and machine (alternating the runs, and taking the median, as the time varies between runs), or count the instructions with `valgrind --tool=callgrind`, which does not depend on the load of the machine. For profiling, build with `DEBUGFLAGS='$(DEBUGFLAGS_PROF)'` (gprof) or use `perf`.

## 12.2 Link-time optimization

As the library is a static library, its functions are not inlined into the program by default. Compiling the program and the library with `-flto` (as the `DEBUGFLAGS_NDEBUG` of Lumipore does) allows the compiler to inline small functions of the library (e.g. those called for each read) into the loops of the program, as if they were in the same file. The vendored library is built with the `DEBUGFLAGS` of the project, and rebuilt automatically when they change (e.g. after a build of the library alone with other flags, by `make test` in its directory).

# 13 Extending the library

## 13.1 What belongs in the library

A module belongs in libcbaia if it is **generic** (useful to more than one program, or likely to be) and needs **no further dependencies** than the C standard library, POSIX, zlib and khash. Examples: file formats of sequencing data, data structures, the handling of barcodes, statistics of reads. Code specific to one tool, or which needs other libraries (an aligner, a YAML parser, htslib), stays in the tool.

The modules of the `seq` component for output files, barcodes, cells and library complexity, the reads reader, and the directory and string helpers were written for Lumipore, and moved to the library when they proved generic; this is the usual path: code is written for a project, and moved when a second project needs it, or when its generality is clear.

## 13.2 Adding a module

1. Choose the component (`core`, `ds`, `io`, `seq`) and a name which is unique over all components and the projects (the headers are included by their name).
2. Write the header: the description of the module, the opaque type, and for each function its documentation (what it does, the parameters, the ownership of the returned values, the behaviour on errors), and a `<module>_test()` function. Follow the developer guide (`docs/dev_manual.md`): the formatting, the naming, the use of the allocation macros and of the logger.
3. Write the implementation, with the unit tests at the end of the `.c` file.
4. Add the test to `tests/cbaia-tests.c` (an entry of the table of the test groups, and its option in the help), and run `make test`.
5. Add the module to the list of the components in `README.md`, to this manual (the chapter of its component, the list of the modules in *Introduction*, and the diagram of the components, generated by `docs/manual/figures/make_diagrams.py`), and regenerate the API reference (`python3 docs/manual/make_reference.py`).
6. Commit (one module, with its tests, in one commit).

The Makefile compiles all `.c` files of the component directories: nothing needs to be registered.

## 13.3 Moving code from a project

When a module of a project becomes part of the library:

1. In the library repository: add the module (adapting its documentation, which should not refer to the project), its tests, and commit; push.
2. In the project: pull the new version of the library (`git subtree pull --prefix ext/libcbaia https://github.com/baia-ipc/libcbaia main --squash`), delete the module from the project, remove its tests from the test program of the project, and replace any duplicated helper functions by those of the library.
3. Check that the outputs and the performance of the project are unchanged: run its tests, and compare the running time and memory with the previous version on realistic data (the library is built with the flags of the project, see *Building and using the library*).

## 13.4 Development workflow

- The library is developed in its own repository, on the `main` branch; the copies in the projects are not modified directly.
- Each commit is atomic (one logical change, compiling and passing the tests), with a short summary line and, if needed, a description of the reason of the change.
- `make test` must pass (under valgrind); new code compiles without warnings.
- Changes of the interface of a module (e.g. a new parameter) require updating the projects which use it, when they pull the new version.

# 14 API reference

The declarations of the headers of the library (generated from the headers by `docs/manual/make_reference.py`; the documentation of each function is in the header, before its declaration). The unit test functions (`<module>_test()`) and the internal functions (_ or __ prefix, called by macros) are not listed.

## 14.1 Core (core)

**alloc.h**

Memory allocation macros

```
#define XMALLOC(PTR, BUFSIZE)
#define XCALLOC(PTR, NITEMS, ITMSIZE)
#define XREALLOC(PTR, REQSIZE)
#define XCHECKSPACE(PTRPTR, SIZEPTR, REQSIZE)
#define XCHECKSPACEINC(PTR, SIZEPTR, REQSIZE, INC)
#define XCHECKSPACEINCPERC(PTR, SIZEPTR, REQSIZE, INCPERC)
#define XFREE(PTR)
```

**argparse-def.h**

```
#define VERSION
#define STR(S)
#define XSTR(S)
#define BOOLSTR(BOOL)
#define HELP_AND_EXIT
#define PURPOSE_AND_EXIT
#define VERSION_AND_EXIT
#define LOGGER_OPTS_VAR_DECLARE
#define PARSE_START
#define PARSE_ERR(FMT, ...)
#define OPT_ERR(C, FMT, ...)
#define ARG_ERR(N, FMT, ...)
static inline bool argparse_long(const char *s, long *value);
#define ARGPARSE_STORE(DESTPTR, VALUE)
#define PARSE_INTEGER_OPT_MIN(C, DESTPTR, MINVALUE)
```

(continues on next page)

(continued from previous page)

```
#define PARSE_INTEGER_OPT_MIN_MAX(C, DESTPTR, MINVALUE, MAXVALUE)
#define PARSE_INTEGER_ARG_MIN(N, DESTPTR, MINVALUE)
#define LOGFILE_OPT
#define LOGFILE_OPT_STR
#define LOGFILE_OPT_HELP
#define LOGGER_LONGOPTS
#define COMMON_LONGOPTS
#define LONGOPTS_END
#define LOGGER_OPTS_HELP
#define COMMON_OPTS_HELP
#define LOGGER_OPTS_STR
#define COMMON_OPTS_STR
#define LOGGER_OPTS_CHKMULTI
#define LOGGER_OPTS_PARSE_CASES
#define COMMON_OPTS_PARSE_CASES
#define CHECK_NARGS(EXPECTED)
#define CHECK_MIN_NARGS(EXPECTED)
#define LOGGER_OPTS_APPLY(LOGGER)
#define INIT_LOGGER(LOGGER)
#define PARSE_LOG(FMT, ...)
#define PARSE_LOG_N(FMT, ...)
```

**colors.h**

ANSI colors of the text written to a terminal

```
#define START_COLOR(STREAM, COLOR)
#define STOP_COLOR(STREAM)
#define BLACK
#define RED
#define GREEN
#define YELLOW
#define BLUE
#define MAGENTA
#define CYAN
#define WHITE
#define BRIGHT_BLACK
#define BRIGHT_RED
#define BRIGHT_GREEN
#define BRIGHT_YELLOW
#define BRIGHT_BLUE
#define BRIGHT_MAGENTA
#define BRIGHT_CYAN
#define BRIGHT_WHITE
```

**file.h**

File system interaction

```
#define xfopen(fname, sfx, mode, description, logger)
#define xfclose(file, logger)
```

(continues on next page)

(continued from previous page)

```
void xfunlink(const char *fname, const char *sfx, Logger *logger);
void xfrename(const char *fname_old, const char *sfx_old,
              const char *fname_new, const char *sfx_new,
              Logger *logger);
void read_file(FILE *file, char **buffer, size_t *bufsize, Logger* logger);
#define xfread(PTR, SIZE, NMEMB, STREAM, LOGGER)
#define xfwrite(PTR, SIZE, NMEMB, STREAM, LOGGER)
size_t read_lines_in_buffer(FILE *file, char *buffer, const size_t bufsize,
    char ***lines, size_t *linesbufsize, Logger *logger);
char *compute_nth_filename(const char *basename, const char *sfx, size_t n);
bool xgetline(char **line, size_t *line_size, FILE *file, Logger *logger);
bool xgetdelim(char **line, size_t *line_size, int delim, FILE *file,
               Logger *logger);
#define xfmap(fname, sfx, writable, fsize, fd, logger)
void xfunmap(void *map, size_t fsize, int fd, Logger *logger);
bool file_exists(const char *fname, const char *sfx);
size_t file_size(FILE *file);
void xchksig(const char *signature, const char *expected,
             const char *fname, const char *sfx,
             const char *desc, const char *helpmsg,
             Logger *logger);
#define xfopen_chksig(fname, sfx, mode, description, expected_signature,␣
↪helpmsg, logger)
#define xfopen_putsig(fname, sfx, mode, description, signature, logger)
#define xfmap_chksig(fname, sfx, fsize, fd, expected_signature, filedesc,␣
↪helpmsg, logger)
bool is_directory(const char *path);
void xmakedirs(const char *path, Logger *logger);
char *path_join(const char *dir, const char *name);
char *path_basename(const char *path);
typedef bool (*list_dir_filter_t)(const char *name, const char *path,
    void *data);
char **list_dir(const char *dir, list_dir_filter_t filter, void *data,
    size_t *n, Logger *logger);
```

**logger.h**

Logging library

```
typedef struct Logger Logger;
typedef enum LoggingLevel {
  LOG_ERROR   = 0,
  LOG_WARN    = 1,
  LOG_SUCCESS = 2,
  LOG_PHASE   = 3,
  LOG_INFO    = 4,
  LOG_PARAM   = 5,
  LOG_MEM     = 6,
  LOG_DEBUG   = 7,
  LOG_TRACE   = 8
```

(continues on next page)

(continued from previous page)

```
} LoggingLevel;
Logger* new_logger(FILE *logfile, LoggingLevel level);
void delete_logger(Logger* logger);
#define lprintf(LOGGER, LEVEL, FMT, ...)
void lnprintf(Logger *logger, LoggingLevel level, const char *fmt, ...)
     __attribute__ ((format (printf,3,4)));
#define LCHECK(CONDITION, LOGGER, FMT, ...)
#define fatal_printf(FMT, ...)
#define showmem(logger)
#define TRACE(FMT, ...)
void NTRACE(const char *fmt, ...)\
     __attribute__ ((format (printf,1,2)));
#define DEBUG_TRACE(FMT, ...)
void DEBUG_NTRACE(const char *fmt, ...)
     __attribute__ ((format (printf,1,2)));
#define DEBUG_NTRACE(fmt, ...)
```

**numutils.h**

Numeric utilities: integer division rounding towards minus infinity, the median of a histogram, and the rounding of a value to a "nice" number (for the axes of plots).

```
static inline long floordiv(long a, long b);
size_t hist_median(const size_t *hist, size_t n_bins, size_t bin_width,
    size_t total);
double nice_number(double v);
```

**progress.h**

Progress display in the style of the Python tqdm package, e.g.:

```
#define PROGRESS_MIN_INTERVAL
typedef struct progress_t progress_t;
progress_t* progress_new(FILE *file, const char *description, size_t total);
void progress_update(progress_t *self, size_t n);
void progress_finish(progress_t *self);
size_t progress_count(const progress_t *self);
void progress_delete(progress_t *self);
```

**strbuf.h**

Growable string buffer, for building output text without allocations for each piece (the memory is kept when the buffer is cleared).

```
#define STRBUF_INC
typedef struct {
  char *s;
  size_t len;
  size_t cap;
} strbuf_t;
static inline void strbuf_init(strbuf_t *self);
static inline void strbuf_free(strbuf_t *self);
```

(continues on next page)

(continued from previous page)

```
static inline void strbuf_clear(strbuf_t *self);
static inline void strbuf_append(strbuf_t *self, const char *s,
    size_t len);
static inline void strbuf_append_str(strbuf_t *self, const char *s);
static inline void strbuf_append_char(strbuf_t *self, char c);
static inline char *strbuf_reserve(strbuf_t *self, size_t n);
static inline void strbuf_commit(strbuf_t *self, size_t n);
static inline void strbuf_append_long(strbuf_t *self, long value);
```

**strutils.h**

```
char *strcat_alloc(const char *s1, const char *s2);
char *xstrdup(const char *s);
bool str_has_suffix(const char *s, const char *sfx);
void fputs_json_string(const char *s, FILE *f);
void fputs_html_escaped(const char *s, FILE *f);
void format_duration(double seconds, char *buf, size_t size);
bool parse_duration(const char *s, double default_unit, long *seconds);
static inline bool is_space(char c);
```

**test.h**

```
#define test_assert(TEST)
#define test_eq(FOUND, EXP, TYPE, FMT)
#define test_eq_size(FOUND, EXP)
#define test_eq_int(FOUND, EXP)
#define test_eq_long(FOUND, EXP)
#define test_eq_str(FOUND, EXP)
#define test_eq_char(FOUND, EXP)
```

## 14.2 Data structures (ds)

**bitwise.h**

```
#define GET_BIT(E, BITNUM)
#define SET_BIT(E, BITNUM)
#define UNSET_BIT(E, BITNUM)
```

**bloom-filter.h**

Bloom filter for integer keys (two hash functions, derived from the splitmix64 hash of the key).

```
typedef struct bloom_filter_t bloom_filter_t;
bloom_filter_t *bloom_filter_new(size_t size_exp);
void bloom_filter_insert(bloom_filter_t *self, size_t key);
bool bloom_filter_query(const bloom_filter_t *self, size_t key);
void bloom_filter_delete(bloom_filter_t *self);
```

**heap.h**

Binary min-heap of pointers, ordered by a comparison function (the smallest element is extracted first), e.g. for priority queues or for merging sorted sequences.

```
typedef int (*heap_cmp_t)(const void *a, const void *b);
typedef struct heap_t heap_t;
heap_t* heap_new(heap_cmp_t cmp, size_t capacity);
void heap_push(heap_t *self, void *element);
void *heap_pop(heap_t *self);
void *heap_peek(const heap_t *self);
size_t heap_size(const heap_t *self);
bool heap_is_empty(const heap_t *self);
void heap_delete(heap_t *self);
```

**hyperloglog.h**

HyperLogLog: estimation of the number of distinct elements of a multiset, using a small fixed amount of memory: the estimator and its small-range correction (linear counting, for estimates up to 2.5 times the number of registers) of Flajolet et al. 2007; 64-bit hash values, as in Heule et al. 2013, so that no large-range correction is needed.

```
#define HLL_PRECISION
#define HLL_REGISTERS
typedef struct {
  uint8_t registers[HLL_REGISTERS];
} hll_t;
void hll_init(hll_t *self);
uint64_t hll_hash(const void *data, size_t len);
void hll_add(hll_t *self, uint64_t hash);
void hll_merge(hll_t *self, const hll_t *other);
double hll_estimate(const hll_t *self);
```

**set.h**

Implementation: - Sorted array S plus an unsorted buffer B - Query is by binary search on S plus linear search on B - Insertion is in B - B is sorted and merged in S when full

```
typedef size_t SET_TYPE;
typedef struct Set Set;
Set *set_new(size_t insertion_buffer_size, size_t bloom_filter_size,
             size_t k);
void set_insert_simple(Set *set, SET_TYPE key);
void set_insert(Set *set, SET_TYPE key);
void set_finalize_insertions(Set *set);
bool set_query(const Set *set, SET_TYPE key);
void set_delete(Set *set);
```

## 14.3 Large files and compression (io)

**bgzf-compressor.h**

BGZF compression (the format of BAM files): the data are compressed in blocks of at most 64 KiB, each a gzip member with an extra field (BC) containing the size of the block. As for the gzip members

of gzip-compressor.h, blocks can be compressed in parallel and written in order; a BGZF file ends with an empty block (the end-of-file marker).

```
#define BGZF_EOF_LEN
extern const uint8_t bgzf_eof[BGZF_EOF_LEN];
typedef struct bgzf_compressor_t bgzf_compressor_t;
bgzf_compressor_t* bgzf_compressor_new(int level);
void bgzf_compressor_run(bgzf_compressor_t *self, const char *data,
    size_t len, strbuf_t *out);
void bgzf_compressor_delete(bgzf_compressor_t *self);
```

**file-merger.h**

Merging of sorted files

```
typedef bool (*merge_getelem_f)(char **elem, size_t *elemsize, FILE *file,
    Logger *logger);
typedef void (*merge_writeelem_f)(FILE *file, char *elem, Logger *logger);
typedef int (*merge_cmpelems_f)(const void *elem1, const void *elem2);
void merge_n_files(size_t nfiles, const char *basename, const char *sfx,
    bool use_stdout, merge_getelem_f getelem, merge_writeelem_f writeelem,
    merge_cmpelems_f cmpelems, Logger *logger);
```

**file-sorter.h**

Sorting of large files in parts which fit in memory (to be merged with merge_n_files() of file-merger.h)

```
size_t sort_file_parts(
    const char *infname,
    const char *infsfx,
    const char *outfname,
    const char *outfsfx,
    size_t bufsize,
    size_t elems_bufsize_init,
    size_t read_elems(FILE *file, char *buffer, size_t bufsize,
                      char ***elems, size_t *elemsbufsize, Logger *logger),
    void sort_elems(char **elems, size_t nelems),
    void output_sorted_elems(FILE *file, char **elems, size_t nelems,
                             Logger *logger),
    Logger *logger);
```

**gzip-compressor.h**

Compression of data blocks as gzip members; the concatenation of gzip members is a valid gzip file, thus blocks can be compressed in parallel and written in order.

```
typedef struct gzip_compressor_t gzip_compressor_t;
gzip_compressor_t* gzip_compressor_new(int level);
void gzip_compressor_run(gzip_compressor_t *self, const char *data,
    size_t len, strbuf_t *out);
void gzip_compressor_delete(gzip_compressor_t *self);
```

**html-page.h**

Self-contained HTML reports: a page with a style for light and dark themes (no external resources), tiles with the main values, and plots as inline SVG (horizontal bar chart, histogram, rank plot).

```
extern const char *const html_page_css;
#define HTML_PAGE_MIN_RELOAD
void html_page_start(html_writer_t *w, const char *title, long refresh);
typedef struct {
  const char *src, *src_dark, *alt;
  int height;
} html_logo_t;
void html_page_start_logo(html_writer_t *w, const char *title,
    const html_logo_t *logo, const char *heading, long refresh);
void html_page_head_end(html_writer_t *w);
void html_dashboard_start(html_writer_t *w, const char *title,
    const html_logo_t *logo, const char *heading, long refresh);
void html_dashboard_bar_end(html_writer_t *w);
void html_panel_start(html_writer_t *w, const char *title, const char *cls);
void html_panel_end(html_writer_t *w);
void html_page_end(html_writer_t *w, const char *footer);
void html_tile(html_writer_t *w, const char *value, const char *label);
typedef struct {
  char s[32];
} html_num_t;
html_num_t html_num(size_t n);
double html_percent(size_t a, size_t b);
void html_hbars(html_writer_t *w, size_t n, const char *const *labels,
    const size_t *values, const char *const *classes, size_t total,
    double mean);
void html_histogram(html_writer_t *w, const size_t *hist, size_t n_bins,
    size_t bin_width, const char *what, const char *unit,
    const char *empty);
void html_knee(html_writer_t *w, const uint64_t *counts, size_t n,
    size_t n_selected, const char *x_label, const char *y_label,
    const char *selected_label);
```

**html-template.h**

HTML templates: static markup (e.g. the skeleton of a page) written as a normal, indented HTML file, embedded in the program at compile time (dev/embed.sh), with named placeholders filled when it is written:

```
#define HTML_TEMPLATE_MAX_VALUES
#define HTML_TEMPLATE_MAX_NESTING
#define html_template(W, TMPL, ...)
void html_template_values(html_writer_t *w, const char *tmpl,
    const char *const *kv, size_t n);
```

**html-writer.h**

Writing HTML (and inline SVG) element by element: the text and the attribute values are escaped in one place, and the output is indented, one element per line, so that the nesting is visible both in the code and in the page.

```c
#define HTML_WRITER_MAX_DEPTH
typedef struct {
  FILE *f;
  const char *tags[HTML_WRITER_MAX_DEPTH];
  unsigned char content[HTML_WRITER_MAX_DEPTH];
  size_t depth;
  size_t inline_from;
  size_t svg;
  const char *start;
  bool start_void;
} html_writer_t;
void html_writer_init(html_writer_t *w, FILE *f);
void html_writer_end(html_writer_t *w);
void html_open(html_writer_t *w, const char *tag);
void html_elem(html_writer_t *w, const char *tag);
void html_attr(html_writer_t *w, const char *name, const char *fmt, ...)
  __attribute__((format(printf, 3, 4)));
void html_inline(html_writer_t *w);
void html_text(html_writer_t *w, const char *text);
void html_textf(html_writer_t *w, const char *fmt, ...)
  __attribute__((format(printf, 2, 3)));
void html_vtextf(html_writer_t *w, const char *fmt, va_list ap);
void html_raw(html_writer_t *w, const char *markup);
void html_close(html_writer_t *w);
void html_tag_text(html_writer_t *w, const char *tag, const char *text);
void html_tag_textf(html_writer_t *w, const char *tag, const char *fmt, ...)
  __attribute__((format(printf, 3, 4)));
size_t html_begin_lines(html_writer_t *w);
```

**md5.h**

MD5 message digest (RFC 1321), e.g. for the checksums of files in metadata (as the md5 of the onlists of seqspec files) or for checking downloads. Not to be used for security purposes.

```c
#define MD5_H
#define MD5_HEX_LEN
typedef struct {
  uint32_t state[4];
  uint64_t n_bytes;
  uint8_t buffer[64];
} md5_t;
void md5_init(md5_t *self);
void md5_update(md5_t *self, const void *data, size_t len);
void md5_final(md5_t *self, char *hex);
void md5_file(const char *path, char *hex, Logger *logger);
```

## 14.4 Sequences and reads (seq)

**bam-reader.h**

Reading of BAM files, e.g. the unaligned BAM files written by Dorado and MinKNOW, in batches of FASTQ-like records (see fastq-reader.h).

```
typedef struct bam_reader_t bam_reader_t;
bool bam_reader_is_bam(const char *filename, Logger *logger);
bam_reader_t* bam_reader_new(const char *filename, Logger *logger);
const char *bam_reader_header(const bam_reader_t *self);
size_t bam_reader_read_batch(bam_reader_t *self, fastq_batch_t *batch,
    size_t max_records);
void bam_reader_delete(bam_reader_t *self);
bool bam_reader_has_eof_marker(const char *filename);
size_t bam_reader_count(const char *filename, Logger *logger);
```

**cell-caller.h**

Identification of the cell barcodes of a single-cell library: the barcodes of a whitelist (e.g. all barcodes of a platform) found exactly in the reads are counted, and the barcodes of the cells are selected from the counts, given the expected number of cells N, as in Cell Ranger (version 2): the barcodes with a count of at least 1/10 of the 99th percentile of the counts of the N most frequent barcodes; as in Cell Ranger, the threshold is rounded to the nearest integer (halves upwards; e.g. 14 for a percentile of 135..144), and is at least 1.

```
typedef struct cell_caller_t cell_caller_t;
cell_caller_t* cell_caller_new(void);
void cell_caller_add(cell_caller_t *self, uint64_t barcode);
void cell_caller_merge(cell_caller_t *self, cell_caller_t *other);
whitelist_t* cell_caller_run(const cell_caller_t *self,
    size_t expected_cells, size_t length, const whitelist_t *known,
    const char *path, Logger *logger);
size_t cell_caller_sorted_counts(const cell_caller_t *self,
    uint64_t **counts);
void cell_caller_delete(cell_caller_t *self);
```

**complexity.h**

Library complexity of a sample: estimation of the number of distinct molecules of the library, from the number of reads and of distinct molecule keys (e.g. the UMI and the first and last bases of the insert, which identify the copies of a molecule without an alignment), for deciding whether deeper sequencing of a library is worthwhile.

```
#define COMPLEXITY_MIN_READS
#define COMPLEXITY_MIN_ERROR_FREE
typedef struct {
  bool estimated;
  bool high_error_rate;
  double error_free;
  double molecules_seen;
  double duplication;
  bool saturated_unknown;
```

(continues on next page)

(continued from previous page)

```
  double molecules;
  double molecules_2x;
} complexity_t;
void complexity_estimate(size_t reads, double distinct_keys,
    double mean_key_length, double error_rate, complexity_t *out);
```

#### edit-distance.h

Bounded edit distance of short sequences (e.g. indices, barcodes, UMIs, compared with the sequences of a list): dynamic programming in the diagonal band of width 2k + 1 around the main diagonal, stopping as soon as a row exceeds the maximum distance k; for sequences of at most SHORT_EDIT_DISTANCE_MAX bases (for longer sequences, use e.g. edlib).

```
#define SHORT_EDIT_DISTANCE_MAX
static inline long short_edit_distance(const char *a, size_t alen,
    const char *b, size_t blen, long k);
static inline long short_prefix_distance(const char *query, size_t qlen,
    const char *target, size_t tlen, long k, size_t *end);
```

#### fastq-reader.h

Reading of FASTQ files (optionally gzip-compressed) in batches of records.

```
typedef struct fastq_reader_t fastq_reader_t;
typedef struct {
  size_t title_from, title_len;
  size_t seq_from, seq_len;
  size_t qual_from;
} fastq_record_t;
typedef struct {
  strbuf_t data;
  fastq_record_t *records;
  size_t n_records, records_cap;
} fastq_batch_t;
fastq_reader_t* fastq_reader_new(const char *filename, Logger *logger);
size_t fastq_reader_read_batch(fastq_reader_t *self, fastq_batch_t *batch,
    size_t max_records);
void fastq_reader_delete(fastq_reader_t *self);
size_t fastq_reader_count(const char *filename, Logger *logger);
void fastq_batch_init(fastq_batch_t *batch);
void fastq_batch_free(fastq_batch_t *batch);
```

#### fastq-writer.h

Writing of FASTQ records to a buffer (e.g. for a batch of records, which is then compressed with gzip-compressor.h and written to a file).

```
void fastq_writer_record(strbuf_t *out, const char *title, size_t title_len,
    const char *seq, const char *qual, size_t len);
```

#### minknow-metadata.h

Read metadata of MinKNOW: the FASTQ files written by MinKNOW contain the read metadata in

the title as fields key=value, separated by spaces (e.g. runid=... ch=12 start_time=...), while Dorado writes SAM tags (see sam-tags.h). The fields can be converted to SAM tags: runid: ri:Z, read: rn:i, ch: ch:i, start_time: st:Z, flow_cell_id: PU:Z, protocol_group_id: pg:Z, sample_id: si:Z, barcode: ob:Z, barcode_alias: oa:Z, parent_read_id: pi:Z, basecall_model_version_id: bm:Z.

```
#define MINKNOW_BARCODE_SAM_TAG
bool minknow_metadata_append_sam_tag(strbuf_t *out, const char *field,
    size_t len);
```

**outfiles.h**

Output files of a program writing the reads to several destinations, e.g. a demultiplexer (a destination for each sample), with one compressed file for each destination: - for an input file: <outdir>/<input basename>.<destination>.fastq.gz, where the suffix of the input file (.fastq, .fq, optionally followed by .gz, or .bam; see reads_reader_is_reads_filename) is removed; - for a group of input files (directory mode): <outdir>/<group>/<destination>.fastq.gz.

```
#define OUTFILES_COMPRESSLEVEL
typedef struct outfiles_t outfiles_t;
outfiles_t* outfiles_new(const char *outdir, const char *reads_file,
    size_t n_destinations, Logger *logger);
outfiles_t* outfiles_new_group(const char *outdir, const char *group,
    size_t n_destinations, Logger *logger);
void outfiles_set_append(outfiles_t *self, bool append);
void outfiles_set_bam(outfiles_t *self, size_t destination,
    const char *header, size_t len);
#define OUTFILES_FASTQ_SFX
#define OUTFILES_BAM_SFX
void outfiles_write(outfiles_t *self, size_t destination, const char *name,
    const char *data, size_t len);
void outfiles_delete(outfiles_t *self);
```

**reads-reader.h**

Reader of the reads of a FASTQ file (optionally gzipped) or of an unaligned BAM file, recognized from the content, with the same interface (batches of FASTQ records, see fastq-reader.h); functions for recognizing the files of reads by their name, counting the reads, and checking whether a file is complete (e.g. when it is written by another program, such as a basecaller).

```
typedef struct reads_reader_t reads_reader_t;
reads_reader_t* reads_reader_new(const char *filename, Logger *logger);
bool reads_reader_is_bam(const reads_reader_t *self);
size_t reads_reader_read_batch(reads_reader_t *self, fastq_batch_t *batch,
    size_t max_records);
void reads_reader_delete(reads_reader_t *self);
bool reads_reader_is_reads_filename(const char *name);
size_t reads_reader_filename_suffix_len(const char *name);
size_t reads_reader_count(const char *filename, Logger *logger);
bool reads_reader_file_complete(const char *path);
```

**sam-header.h**

SAM header text of BAM files written from BAM input (e.g. the unaligned BAM of Dorado), keeping the provenance of the reads: - the read groups (@RG) of the inputs, so that the records keep the RG

tag of their input; a program which distributes the reads to several destinations (e.g. a demultiplexer) can rename them per destination; - the programs (@PG) of the inputs, followed by the @PG line of the program, linked to the last one of the inputs (PP), as samtools does.

```
typedef struct {
  strbuf_t rg_lines, pg_lines;
  char **rg_ids;
  size_t n_rg;
  char *last_pg;
} sam_header_inputs_t;
void sam_header_inputs_init(sam_header_inputs_t *self);
void sam_header_inputs_clear(sam_header_inputs_t *self);
void sam_header_inputs_free(sam_header_inputs_t *self);
void sam_header_inputs_add(sam_header_inputs_t *self, const char *text);
void sam_header_inputs_add_files(sam_header_inputs_t *self,
    char *const *files, size_t n_files, Logger *logger);
void sam_header_append_read_groups(strbuf_t *text,
    const sam_header_inputs_t *inputs, const char *name);
void sam_header_append_programs(strbuf_t *text,
    const sam_header_inputs_t *inputs, const char *program,
    const char *version, const char *command_line);
char *sam_header_command_line(int argc, char *const argv[]);
const char *sam_header_line_id(const char *line, size_t line_len,
    size_t *len);
```

**sam-tags.h**

SAM tags in text format, in read titles: the title of a read (FASTQ header without '@', or the title returned by the BAM reader) is the read name followed by fields; in the FASTQ written by Dorado, and in the output of programs which keep the tags (e.g. for minimap2 -y or samtools import -T '*'), the fields are SAM tags in text format (e.g. ch:i:12), separated by tabs. The uBAM writer encodes them in binary form. For the read metadata of MinKNOW (fields key=value), see minknow-metadata.h.

```
bool sam_tag_is_valid(const char *field, size_t len);
void sam_tag_start(strbuf_t *out, const char *tag, char type);
void sam_tag_append_string(strbuf_t *out, const char *tag,
    const char *value, size_t len);
void sam_tag_append_int(strbuf_t *out, const char *tag, long value);
```

**sequtil.h**

Sequence utilities: reverse complement and substrings with the semantics of the Python slices (useful for reproducing exactly the output of Python programs).

```
void reverse_complement(const char *seq, size_t len, char *dest);
void py_slice(long len, long start, long stop, long *from, long *slice_len);
```

**ubam-writer.h**

Encoding of unaligned BAM data (header and records), to be compressed with BGZF (see bgzf-compressor.h).

```
#define UBAM_MAX_READ_NAME
void ubam_writer_header(strbuf_t *out, const char *text);
void ubam_writer_record(strbuf_t *out, const char *title, size_t title_len,
    const char *seq, const char *qual, size_t len);
```

**whitelist.h**

Whitelist of barcodes (e.g. the cell barcodes of a single-cell platform) and correction of the barcodes extracted from the reads.

```
#define WHITELIST_MAX_BARCODE_LENGTH
#define WHITELIST_MAX_DIST
typedef struct whitelist_t whitelist_t;
typedef enum {
  WHITELIST_EXACT,
  WHITELIST_CORRECTED,
  WHITELIST_AMBIGUOUS,
  WHITELIST_NOT_FOUND,
} whitelist_status_t;
typedef struct {
  whitelist_status_t status;
  uint64_t barcode;
  long dist;
  size_t consumed;
} whitelist_match_t;
whitelist_t* whitelist_new(const char *path, Logger *logger);
whitelist_t* whitelist_new_from_codes(const uint64_t *barcodes, size_t n,
    size_t length);
bool whitelist_encode(const whitelist_t *self, const char *seq,
    uint64_t *barcode);
bool whitelist_contains(const whitelist_t *self, uint64_t barcode);
void whitelist_delete(whitelist_t *self);
size_t whitelist_barcode_length(const whitelist_t *self);
size_t whitelist_size(const whitelist_t *self);
size_t whitelist_codes(const whitelist_t *self, uint64_t **codes);
void whitelist_match(const whitelist_t *self, const char *seq, size_t len,
    bool from_end, long max_dist, whitelist_match_t *match);
void whitelist_decode(const whitelist_t *self, uint64_t barcode, char *out);
```